

DSA: Graphs & Trie - తెలుగు గైడ్ (LeetCode 150, SSE)

ఈ document చదివిన తర్వాత Graph మరియు Trie problems మళ్ళీ జీవితంలో మర్చిపోకూడదు. ప్రతి problem కి – ఎలా ఆలోచించాలి (intuition first), ఒక vivid real-life analogy, ఇచ్చిన input ని **graph గా ఎలా చూడాలి** (nodes ఏవి, edges ఏవి), naive నుండి optimal వరకు thought process, clean commented JavaScript solution, పెన్సిల్తో గీసినట్టు dry run, complexity reasoning (V, E terms లో), pattern takeaway, మరియు edge cases (disconnected components, cycles, visited-marking) – అన్నీ ఉంటాయి.

లక్ష్యం: DSA అస్సలు తెలియని person ని – ఎవరికైతే graph అంటే ఏమిటో, BFS/DFS అంటే ఏమిటో కూడా తెలియదో – వాళ్ళని SSE (Senior Software Engineer) interview లో confident గా ఈ 12 problems solve చేసేలా తయారు చేయడం. మనం facts బట్టి పట్టడం కాదు – **ఎలా ఆలోచించాలో** నేర్చుకుంటాం. మూడు core patterns (Graph traversal, Topological Sort, Trie) అర్థమైతే, ఈ 12 మాత్రమే కాదు, వీటిలాంటి 100 problems కూడా solve చేయగలము.

గమనిక: Big-O notation, arrays, recursion, hash map / hash set basics, queue, stack, time/space complexity అంటే ఏమిటి – ఇలాంటి పునాదులు (fundamentals) [DSA_00_Foundations_Telugu.md](#) లో ఉన్నాయి. అవి ముందు చదివితే ఈ guide ఇంకా సులభంగా అర్థమవుతుంది. ఇక్కడ మనం నేరుగా Graph & Trie patterns లోకి దిగుతాం.

విషయ సూచిక (Table of Contents)

Pattern Primers (ముందు వీటిని చదువు – problems కి foundation)

- Pattern: Graph DFS / BFS (adjacency list, visited set, grid-as-graph, connected components / islands template)
- Pattern: Topological Sort (Kahn's BFS with indegree + DFS with cycle detection)
- Pattern: Trie (prefix tree – node structure, insert, search)

Part 1 – Graph (General: DFS / Union / Topo)

1. Number of Islands (LeetCode #200) – Medium
2. Surrounded Regions (LeetCode #130) – Medium
3. Clone Graph (LeetCode #133) – Medium
4. Evaluate Division (LeetCode #399) – Medium
5. Course Schedule (LeetCode #207) – Medium
6. Course Schedule II (LeetCode #210) – Medium

Part 2 – Graph (BFS shortest path)

7. Snakes and Ladders (LeetCode #909) – Medium
8. Minimum Genetic Mutation (LeetCode #433) – Medium
9. Word Ladder (LeetCode #127) – Hard

Part 3 — Trie (Prefix Tree)

10. Implement Trie (Prefix Tree) (LeetCode #208) — Medium
11. Design Add and Search Words Data Structure (LeetCode #211) — Medium
12. Word Search II (LeetCode #212) — Hard

Pattern: Graph DFS / BFS

వివరణ

Graph అంటే ఒక data structure — కొన్ని **nodes** (వీటిని vertices అంటారు) మరియు వాటిని కలిపే **edges** (connections). అంతే. చుట్టూ చూడు — మనుషుల మధ్య friendships, cities మధ్య roads, web pages మధ్య links, courses మధ్య prerequisites — అన్నీ graphs.

మనం గుర్తుంచుకోవాల్సిన కొన్ని పదాలు:

- **Directed vs Undirected:** edge కి direction ఉందా? "A follows B" (Twitter) = **directed** ($A \rightarrow B$ అంటే $B \rightarrow A$ కాదు). "A is friends with B" (Facebook) = **undirected** (రెండువైపులా).
- **Weighted vs Unweighted:** edge మీద ఒక number (cost/distance) ఉందా? Road కి distance ఉంది = weighted. Simple friendship = unweighted.
- **Cyclic vs Acyclic:** ఒక node నుండి బయలుదేరి, edges వెంట వెళ్తూ, తిరిగి అదే node కి రాగలమా? వస్తే **cycle** ఉంది. **DAG** = Directed Acyclic Graph (cycle లేని directed graph) — topological sort కి కీలకం.

Graph తో మనం చేసే మొదటి పని ఎప్పుడూ **traversal** — అన్ని nodes ని క్రమపద్ధతిలో ఒకసారి visit చెయ్యడం. రెండు రకాలు:

- **DFS (Depth-First Search):** ఒక దారి పట్టుకుని **చివరిదాకా లోతుగా** వెళ్ళు, dead-end వచ్చాక వెనక్కి వచ్చి (backtrack) వేరే దారి చూడు. Recursion (లేదా stack) తో చేస్తాం.
- **BFS (Breadth-First Search):** ప్రస్తుత node నుండి **దగ్గరి neighbors ని అందరినీ** ముందు చూడు, తర్వాత వాళ్ళ neighbors ని — ring by ring, level by level. Queue తో చేస్తాం. **Unweighted graph లో shortest path** కావాలంటే BFS దే రాజ్యం.

Real-life Scenario

DFS = ఒక maze (చిక్కుదారి) లో ఒంటరిగా. ఒక చేతిని గోడ మీద పెట్టి, ఒక దారి పట్టుకుని ముందుకెళ్తూ ఉంటావు — dead end వచ్చేదాకా. Dead end వచ్చాక, చివరి కూడలి (junction) దగ్గరికి వెనక్కి వచ్చి, ఇంకో దారి try చేస్తావు. "ముందు లోతుకి వెళ్ళు, తర్వాత వెనక్కి" — అదే DFS.

BFS = చెరువులో రాయి వేస్తే వచ్చే అలలు (ripples). రాయి పడిన చోటి నుండి అలలు **వలయాలుగా (rings)** బయటికి వ్యాపిస్తాయి — ముందు దగ్గరివి, తర్వాత దూరంవి. నీ friend ద్వారా, ఆ friend యొక్క friend ద్వారా — "ఎన్ని handshakes లో ఒకరిని చేరతాను" (degrees of separation) అనే ప్రశ్న BFS దే. దగ్గరివాళ్ళని ముందు చేరతాం కాబట్టి, మొదట చేరిన దారే **shortest** (అతి తక్కువ steps).

గ్రాఫ్‌ని code లో ఎలా represent చేయాలి

మూడు రూపాలు తరచూ కనిపిస్తాయి. వేటిని ఎప్పుడు వాడాలో తెలుసుకో:

Representation	ఎప్పుడు	JS లో
Adjacency List	చాలావరకు ఇదే. Sparse graphs (edges తక్కువ)	Map లేదా Array<Array> – node → దాని neighbors list
Adjacency Matrix	Dense graphs, "edge ఉందా?" O(1) లో కావాలి	n × n 2D array, matrix[u][v] = 1
Implicit / Grid	Node ని explicit గా ఇవ్వరు – grid cells, strings	Cell (r, c) నే node; పక్క cells నే neighbors

కీలక realization: చాలా graph problems లో "graph" అనే మాటే ఉండదు! Grid ఇస్తారు, లేదా strings list ఇస్తారు. నీ పని ఆ input ని graph గా **అనువదించడం** – "nodes ఏవి? edges ఏవి?" అని అడగడం. ఇదే SSE interview లో నిన్ను గెలిపించే skill.

Adjacency List build (template)

```
// edges = [[u, v], ...] (undirected అనుకుందాం)
function buildGraph(n, edges) {
  // Array of arrays – node 0..n-1 కి ఒక్కో list
  const graph = Array.from({ length: n }, () => []);
  for (const [u, v] of edges) {
    graph[u].push(v);
    graph[v].push(u); // undirected కాబట్టి రెండువైపులా. directed అయితే ఈ line తీసేయి.
  }
  return graph;
}

// Nodes strings/labels అయితే Map వాడు:
function buildGraphMap(edges) {
  const graph = new Map();
  const add = (u, v) => {
    if (!graph.has(u)) graph.set(u, []);
    graph.get(u).push(v);
  };
  for (const [u, v] of edges) { add(u, v); add(v, u); }
  return graph;
}
```

DFS & BFS templates

```
// ----- DFS (recursive) -----
function dfs(graph, node, visited = new Set()) {
  if (visited.has(node)) return; // ⚠ ఇదే అతి ముఖ్యమైన line – లేకపోతే infinite loop
  visited.add(node); // "వచ్చాను" అని గుర్తు పెట్టు
  // ... ఇక్కడ node ని process చెయ్యి ...
  for (const nb of graph.get(node) ?? []) {
    dfs(graph, nb, visited); // ప్రతి neighbor లోకి లోతుగా
  }
}
```

```

    }
  }

  // ----- BFS (iterative, queue) -----
  function bfs(graph, start) {
    const visited = new Set([start]);
    const queue = [start]; // JS array ని queue లా వాడతాం
    while (queue.length) {
      const node = queue.shift(); // ముందు వచ్చినది ముందు (FIFO)
      // ... node ని process చెయ్యి ...
      for (const nb of graph.get(node) ?? []) {
        if (!visited.has(nb)) {
          visited.add(nb); // ⚠️ push చేసేటప్పుడే mark చెయ్యి (dequeue వద్ద కాదు)
          queue.push(nb); // లేకపోతే ఒకే node multiple సార్లు queue లోకి వెళ్తుంది
        }
      }
    }
  }
}

```

Grid ని graph లా చూడటం (Islands / flood fill template)

Grid problems లో ప్రతి cell (r, c) ఒక node. దాని neighbors = **4 దిక్కులు** (up/down/left/right) cells. (కొన్ని problems లో 8 దిక్కులు – diagonals తో.) "Connected component" = ఒకదానితో ఒకటి connect అయిన cells గుంపు (ఒక island).

```

// Grid లో connected components (islands) లెక్కించే universal template
function countComponents(grid, isLand) {
  const rows = grid.length, cols = grid[0].length;
  let count = 0;

  const dfs = (r, c) => {
    // హద్దులు దాటితే, లేదా land కాకపోతే, లేదా ఇప్పటికే చూసి ఉంటే → ఆగు
    if (r < 0 || r >= rows || c < 0 || c >= cols || !isLand(grid[r][c])) return;
    grid[r][c] = '0'; // visited గా mark (ఇక్కడ నీటిగా మార్చడం = visited set బదులు)
    dfs(r + 1, c); dfs(r - 1, c); dfs(r, c + 1); dfs(r, c - 1); // 4 దిక్కులు
  };

  for (let r = 0; r < rows; r++) {
    for (let c = 0; c < cols; c++) {
      if (isLand(grid[r][c])) { // కొత్త land కనిపిస్తే అది కొత్త island మొదలు
        count++;
        dfs(r, c); // ఆ island మొత్తం "మునిగిపో" (sink) – మళ్ళీ లెక్కించకుండా
      }
    }
  }
  return count;
}

```

గమనిక – visited ను ఎలా track చేయాలి: (1) ఒక Set / boolean matrix వాడు, లేదా (2) grid ని **in-place modify** చెయ్యి (land ని water గా మార్చడం). రెండోది extra space ఆదా చేస్తుంది కానీ input ని పాడు చేస్తుంది – interviewer అడిగితే "grid ని modify చేయొచ్చా?" అని clarify చెయ్యడం మంచి signal.

ఎలా గుర్తించాలి (Recognition Signals)

ఈ signals కనిపిస్తే graph traversal గురించి ఆలోచించు:

- **Grid / matrix + "connected", "island", "region", "surrounded", "flood"** → DFS/BFS flood fill.
- **"Shortest path", "minimum steps/moves", "fewest transformations"** – edges కి weight లేకపోతే → **BFS** (levels = distance).
- **"Clone / deep copy a graph"**, "cycle ఉందా", "అన్నీ connect అయ్యాయా" → traversal + visited.
- **Elements మధ్య dependency / relationship** ("a/b = 2", "x, y కి connected") → nodes + edges గా model చేసి traverse.
- **DFS ఎప్పుడు, BFS ఎప్పుడు:** shortest/level కావాలంటే BFS. కేవలం "అన్నీ చూడాలి / connectivity / path ఉందా" అయితే ఏదైనా సరే (DFS recursion తో సులభం).

Complexity

- **Time:** $O(V + E)$ – ప్రతి node ఒకసారి visit (V), ప్రతి edge ఒకసారి (లేదా undirected అయితే రెండుసార్లు) traverse (E). ఇదే graph traversal యొక్క golden complexity.
- **Grid కి:** $V = \text{rows} \times \text{cols}$, ప్రతి cell కి 4 edges → $E = O(V)$, కాబట్టి $O(\text{rows} \times \text{cols})$.
- **Space:** $O(V)$ – visited set + recursion stack (DFS) / queue (BFS). Grid worst case లో recursion depth $\text{rows} \times \text{cols}$ (అంతా land అయితే) → stack overflow జాగ్రత్త; పెద్ద grids కి BFS (iterative) safer.

1. Number of Islands (LeetCode #200) – Medium

సమస్య: '1' (land) మరియు '0' (water) characters తో ఉన్న 2D grid grid ఇస్తారు. **islands** సంఖ్య లెక్కించాలి. Island అంటే – horizontally లేదా vertically (4 దిక్కులు, diagonals కాదు) connect అయిన land cells గుంపు. Grid యొక్క నాలుగు అంచులూ నీటితో చుట్టబడి ఉన్నాయని అనుకో.

Constraints: $1 \leq m, n \leq 300$; ప్రతి cell '0' లేదా '1'.

ఉదాహరణ:

Input:

```
11110
11010
11000
00000
```

Output: 1 (అన్ని 1లు ఒకదానితో ఒకటి connect → ఒకే island)

Input:

```
11000
11000
00100
00011
```

Output: 3 (ఎడమ-పైన గుంపు, మధ్యలో ఒక్కటి, కుడి-కింద గుంపు)

ఎలా ఆలోచించాలి:

మొదట ఇది "grid problem" లా కనిపిస్తుంది, కానీ దీన్ని **graph** గా చూడు. ప్రతి land cell (r, c) ఒక **node**. రెండు land cells పక్కపక్కన (4 దిక్కుల్లో) ఉంటే వాటి మధ్య ఒక **edge**. అప్పుడు "island" అంటే ఏమిటి? ఒక **connected component** – ఒకదానితో ఒకటి కలిసిన nodes గుంపు. కాబట్టి ప్రశ్న మారుతుంది: "ఈ graph లో ఎన్ని connected components ఉన్నాయి?"

Connected components లెక్కించడం ఎలా? Grid అంతా scan చెయ్యి. ఒక కొత్త land cell ('1') కనిపిస్తే – అది ఒక **కొత్త island మొదలు** అని అర్థం (ఎందుకంటే మనం ఇప్పటిదాకా దీన్ని చూడలేదు). Count ఒకటి పెంచు. తర్వాత ఆ cell నుండి DFS (లేదా BFS) చేసి, **ఆ island లోని land cells అన్నిటినీ** water గా మార్చేయి (sink చేయి). దీనివల్ల అదే island ని మళ్ళీ లెక్కించం. ఇలా grid అంతా తిరిగితే, count = islands సంఖ్య.

Visited ను track చెయ్యడానికి extra set బదులు, cell ని '1' → '0' గా మార్చడమే simplest (in-place). ఇది "మునిగిపో" (sink the island) technique.

Optimal Approach:

Insight: Grid = implicit graph; island = connected component. Grid scan చేస్తూ, ప్రతి కొత్త '1' కి count++ చేసి DFS తో ఆ component మొత్తం sink చెయ్యి.

Plan:

1. count = 0 . Grid లోని ప్రతి cell మీద rows × cols loop.
2. Cell '1' అయితే → count++ , ఆ cell నుండి dfs(r, c) .
3. dfs : హద్దుల్లో ఉండా + '1' ఆ చూడు; అవును అయితే '0' గా మార్చి, 4 దిక్కుల్లో recurse.
4. Loop అయ్యాక count return.

Solution (JavaScript):

```
function numIslands(grid) {
  if (!grid || grid.length === 0) return 0;
  const rows = grid.length, cols = grid[0].length;
  let count = 0;

  // ఒక land cell నుండి, దానికి connect అయిన మొత్తం island ను "ముంచేయి".
  const sink = (r, c) => {
    // హద్దు దాటితే / నీరు అయితే / ఇప్పటికే ముంచేస్తే → ఆగు (base case)
    if (r < 0 || r >= rows || c < 0 || c >= cols || grid[r][c] !== '1') return;
    grid[r][c] = '0'; // visited గా mark = నీటిగా మార్చు
    sink(r + 1, c); // కింద
    sink(r - 1, c); // పైన
    sink(r, c + 1); // కుడి
    sink(r, c - 1); // ఎడమ
  };

  for (let r = 0; r < rows; r++) {
    for (let c = 0; c < cols; c++) {
      if (grid[r][c] === '1') { // ఇంకా ముంచని కొత్త land → కొత్త island
        count++;
        sink(r, c); // ఈ island మొత్తం ఒకేసారి ముంచేయి
      }
    }
  }
}
```

```
    return count;
}
```

Dry Run: grid = `[["1","1","0"],["0","1","0"],["0","0","1"]]`

```
(0,0)='1' → count=1, sink(0,0):
  (0,0)→'0'; పక్కన (1,0)='0' ఆగు, (0,1)='1'→'0';
  (0,1) నుండి (1,1)='1'→'0'; (1,1) చుట్టూ అంతా '0' → backtrack.
  ఇప్పుడు grid ఎడమ-పై గుంపు అంతా '0'.
(0,1),(0,2)... ='0' skip
(1,1)='0' skip (అప్పటికే ముంచేశాం)
(2,2)='1' → count=2, sink(2,2): (2,2)→'0'; చుట్టూ అంతా హద్దు/నీరు → backtrack.
loop ముగిసింది → return 2
```

Complexity:

- **Time:** $O(m \times n)$ — ప్రతి cell ను ఎక్కువలో ఎక్కువ రెండుసార్లు తాకుతాం (outer loop + sink). $V = m \times n$ nodes, $E = O(m \times n)$ edges → $O(V + E) = O(m \times n)$.
- **Space:** $O(m \times n)$ worst case — recursion stack depth. Grid అంతా land అయితే ("1111...1" ఒకే snake లా) recursion $m \times n$ deep వెళ్తుంది. Extra visited array వాడకపోవడం వల్ల auxiliary data structure space $O(1)$, కానీ call stack $O(m \times n)$.

గుర్తుంచుకోవాల్సినది:

Grid + "connected / island / region" = flood fill (DFS/BFS on implicit graph). ఈ template ఒక్కటి తెలిస్తే వందల problems: **Max Area of Island** (#695 — sink చేస్తూ size లెక్కించు), **Flood Fill** (#733), **Number of Closed Islands**, **Making a Large Island**, **Pacific Atlantic Water Flow** (border నుండి reverse DFS) — అన్నీ ఇదే idea. కీలకం: "ప్రతి కొత్త component కి count++ చేసి, ఆ component మొత్తం visited గా mark చెయ్యి."

సాధారణ తప్పులు:

- **Visited mark మర్చిపోవడం:** cell ను '0' గా మార్చకపోతే, sink() అదే land ని పదేపదే visit చేస్తూ **infinite recursion** → stack overflow. Visited-marking graph traversal యొక్క ప్రాణం.
- **Boundary check తప్పు:** $r \geq \text{rows}$, $c \geq \text{cols}$ (equal కూడా) చూడకపోతే array out-of-bounds. $<$ కాదు $\leq$ వాడితే bug.
- **Diagonals కలపడం:** ఈ problem 4 దిక్కులు మాత్రమే. పొరపాటున 8 దిక్కులు (diagonals) కలిపితే tests fail. Problem statement జాగ్రత్తగా చదువు.
- **In-place modify కి interviewer okనా:** input grid ను పాడుచేస్తున్నాం. Real interview లో "grid modify చేయొచ్చా, లేక untouched గా ఉంచాలా?" అని అడుగు; untouched కావాలంటే separate visited boolean matrix వాడు.
- **పెద్ద grid లో recursion depth:** $300 \times 300 = 90,000$ deep recursion JS లో stack overflow అవ్వొచ్చు. Production/safety కి iterative BFS (queue) కి మారడం.

2. Surrounded Regions (LeetCode #130) — Medium

సమస్య: 'X' మరియు '0' లతో నిండిన 2D board ఇస్తారు. నాలుగు వైపులా 'X' లతో పూర్తిగా చుట్టబడిన '0' regions అన్నిటిని 'X' గా flip చెయ్యాలి. అంటే: ఒక '0' region **board అంచు (border) ని తాకితే** అది "escape" అవుతుంది — దాన్ని flip చేయకూడదు. Board ను in-place modify చెయ్యాలి.

Constraints: $1 \leq m, n \leq 200$; ప్రతి cell 'X' లేదా '0'.

ఉదాహరణ:

Input:		Output:
X X X X		X X X X
X 0 0 X	→	X X X X
X X 0 X		X X X X
X 0 X X		X 0 X X

(మధ్యలో 0 లు పూర్తిగా చుట్టబడ్డాయి → flip. కింద-ఎడమ 0 border ను తాకింది → అలానే ఉంటుంది.)

ఎలా ఆలోచించాలి:

మొదటి ఆలోచన: "ప్రతి '0' region ను DFS చేసి, అది border ను తాకుతుందా చూడు; తాకకపోతే flip." ఇది పని చేస్తుంది కానీ కొంచెం awkward — region ను గుర్తుంచుకుని, border తాకిందే లేదో flag పెట్టి, తర్వాత మళ్ళీ వెళ్ళి flip చెయ్యాలి.

ఇప్పుడు దీన్ని **తిరగేసి (inversion trick)** ఆలోచించు. మనం flip చేయకూడని '0' లు ఏవి? **Border ను తాకేవి** (వాటికి connect అయినవన్నీ). అవి చాలా తక్కువ, గుర్తించడం సులభం. కాబట్టి plan:

- Border cells నుండి మాత్రమే** DFS మొదలు పెట్టు. వాటికి connect అయిన '0' లన్నిటిని ఒక temporary marker ('#') తో గుర్తు పెట్టు — ఇవి "safe / escaped."
- తర్వాత board అంతా తిరుగు: మిగిలిన '0' లు (border ను తాకనివి) → 'X' (flip). '#' లు → తిరిగి '0' (restore).

కీలక realization: "surrounded ను వెతకడం కష్టం; escaped ను వెతకడం సులభం. Escaped కానివన్నీ surrounded." ఇది చాలా graph problems లో పనికొచ్చే **"start from the boundary / reverse thinking"** technique.

Brute Force / naive:

ప్రతి inner '0' region కి separate DFS చేసి, ఆ region border ను తాకుతుందే లేదో boolean తో track చేసి, తాకకపోతే మళ్ళీ వెళ్ళి flip. పని చేస్తుంది కానీ regions ను రెండుసార్లు process చేయాలి, code messy. Complexity same $O(m \times n)$ కానీ border-first approach శుభ్రం, bug తక్కువ.

Optimal Approach:

Insight: Flip కాకూడని '0' = border-connected '0'. వాటిని border DFS తో '#' మార్క్ చెయ్యి. మిగిలినవన్నీ surrounded → flip.

Plan:

- మొదటి/చివరి row, మొదటి/చివరి column లోని ప్రతి '0' నుండి dfs — connect అయిన '0' లన్నీ '#'.
- Board అంతా scan: '0' → 'X'; '#' → '0'.

Solution (JavaScript):

```
function solve(board) {
    if (!board || board.length === 0) return;
    const rows = board.length, cols = board[0].length;

    // ఒక border-connected '0' నుండి, connect అయిన అన్ని '0' లను '#' గా mark చెయ్యి.
    const mark = (r, c) => {
        if (r < 0 || r >= rows || c < 0 || c >= cols || board[r][c] !== '0') return;
        board[r][c] = '#'; // safe/escaped గా గుర్తు
        mark(r + 1, c); mark(r - 1, c); mark(r, c + 1); mark(r, c - 1);
    };

    // 1) అన్ని border cells నుండి మొదలుపెట్టు
    for (let r = 0; r < rows; r++) {
        mark(r, 0); // మొదటి column
        mark(r, cols - 1); // చివరి column
    }
    for (let c = 0; c < cols; c++) {
        mark(0, c); // మొదటి row
        mark(rows - 1, c); // చివరి row
    }

    // 2) మిగిలిన '0' = surrounded → flip; '#' = escaped → restore
    for (let r = 0; r < rows; r++) {
        for (let c = 0; c < cols; c++) {
            if (board[r][c] === '0') board[r][c] = 'X';
            else if (board[r][c] === '#') board[r][c] = '0';
        }
    }
}
```

Dry Run: board = `[["X","X","X"],["X","0","X"],["X","X","X"]]` (మధ్యలో ఒక్క 0)

Border DFS: అన్ని border cells 'X' → mark() వెంటనే return, ఏ '0' కీ '#' రాదు.
Final scan: `(1,1)='0' → border ను తాకలేదు → 'X'.`
Result: అంతా 'X'. ✅ (మధ్య 0 పూర్తిగా చుట్టబడింది)

board = `[["X","0"],["0","X"]]` :

Border DFS: `(0,1)='0'` (చివరి column, border) → '#'. `(1,0)='0'` (మొదటి column, border) → '#'.
Final scan: '#' లు రెండూ → '0' గా restore. '0' మిగిలేదు flip చేయడానికి.
Result: `[["X","0"],["0","X"]]` మారలేదు. ✅ (రెండు 0 లూ border ను తాకాయి → escaped)

Complexity:

- Time:** $O(m \times n)$ — border DFS ప్రతి cell ను ఎక్కువలో ఒకసారి తాకుతుంది, final scan మరోసారి. $O(V + E)$ with $V, E = O(m \times n)$.
- Space:** $O(m \times n)$ worst case — recursion stack (అంతా border-connected '0' అయితే).

గుర్తుంచుకోవాల్సినది:

"Boundary/border నుండి reverse గా ఆలోచించు" – ఇది శక్తివంతమైన pattern. నేరుగా "surrounded" ను వెతకడం కష్టమైనప్పుడు, "surrounded కానివి (border-connected)" ను మార్క్ చేసి, మిగిలినవన్నీ surrounded అని conclude చెయ్యి. అదే idea: **Pacific Atlantic Water Flow** (#417 – ocean అంచుల నుండి reverse DFS), **Number of Enclaves** (#1020), **Number of Closed Islands** (#1254). Temporary marker ('#') వాడి 3-state (X / O / escaped) గా విడగొట్టడం క్లీన్ trick.

సాధారణ తప్పులు:

- **నేరుగా inner 0 లను flip చేయడం:** border check లేకుండా అన్ని '0' లను flip చేస్తే, escaped regions కూడా పోతాయి – తప్పు.
- **Temporary marker ను restore మర్చిపోవడం:** '#' లను తిరిగి '0' గా మార్చకపోతే board లో '#' మిగిలిపోతుంది – తప్పు output.
- **Border cells అన్నీ cover చేయకపోవడం:** 4 అంచులూ (రెండు rows + రెండు columns) loop చెయ్యాలి. మూలలు (corners) రెండు loops లో వస్తాయి కానీ **mark** idempotent కాబట్టి safe.
- **Single row/column board:** $1 \times n$ లేదా $n \times 1$ board లో అన్ని cells border మీదే ఉంటాయి → ఏ '0' flip కాదు. Border-first approach దీన్ని సహజంగా handle చేస్తుంది (special-case అవసరం లేదు).

3. Clone Graph (LeetCode #133) – Medium

సమస్య: ఒక **connected undirected graph** లోని ఒక node reference ఇస్తారు. ఆ graph యొక్క **deep copy (clone)** చేసి, clone లోని corresponding node ను return చెయ్యాలి. ప్రతి node లో val (integer) మరియు neighbors (సక్క nodes list) ఉంటాయి. "Deep copy" అంటే – clone లో ఏ node కూడా original node లను reference చేయకూడదు; పూర్తిగా కొత్త objects.

```
class Node {
    constructor(val, neighbors) {
        this.val = val === undefined ? 0 : val;
        this.neighbors = neighbors === undefined ? [] : neighbors;
    }
}
```

Constraints: nodes $1..100$; $1 \leq val \leq 100$, val unique; graph connected, no self-loops, no repeated edges; input null అయితే null return.

ఉదాహరణ:

Input: adjList = [[2,4],[1,3],[2,4],[1,3]] (node i కి neighbors)
 node 1 ↔ 2, 1 ↔ 4, 2 ↔ 3, 3 ↔ 4 (ఒక square)
 Output: అదే structure తో పూర్తిగా కొత్త nodes.

ఎలా ఆలోచించాలి:

ఇది traversal problem, కానీ ఒక ముఖ్యమైన మెలిక ఉంది. Graph లో **cycles** ఉంటాయి (undirected కాబట్టి $1 \rightarrow 2$ ఉంటే $2 \rightarrow 1$ కూడా ఉంది – అదే ఒక cycle). నువ్వు node 1 ను clone చేస్తూ దాని neighbor 2 ను clone చేస్తావు; 2 కి neighbor 1

ఉంది కాబట్టి మళ్ళీ 1 ను clone చేయబోతావు... **infinite loop!**

దీన్ని ఆపడానికి కీలకం: "**ప్రతి original node ను ఒక్కసారే clone చెయ్యి, ఆ mapping ను గుర్తుంచుకో.**" ఒక **Map** (original node → cloned node) పెట్టు. ఏ node ను clone చేసేముందు, "దీన్ని ఇప్పటికే clone చేశానా?" అని map లో చూడు. చేసి ఉంటే, ఉన్న clone ను వాడు (మళ్ళీ చేయకు). ఇదే visited-tracking + node reuse కలిపిన రూపం.

Traversal DFS (recursion) లేదా BFS (queue) – రెండూ పని చేస్తాయి; map ఉంటే చాలు. DFS recursion శుభ్రంగా ఉంటుంది: "ఈ node ను clone చెయ్యి, తర్వాత దాని ప్రతి neighbor ను (recursively) clone చేసి, clone యొక్క neighbors list కి కలుపు."

Optimal Approach:

Insight: **Map**<original, clone> – visited + copy రెండింటినీ ఒకేసారి చూసుకుంటుంది. Node ను మొదటిసారి కనిపించినప్పుడు clone చేసి **వెంటనే** map లో పెట్టు (neighbors process చేసేముందే – లేకపోతే cycle లో infinite loop).

Plan (DFS):

1. **map = new Map()** .
2. **dfs(node)** : map లో ఉంటే return map value. లేకపోతే కొత్త **Node(node.val)** create, **వెంటనే** **map.set(node, copy)** . తర్వాత ప్రతి neighbor కి **copy.neighbors.push(dfs(nb))** . **copy** return.
3. return **node ? dfs(node) : null** .

Solution (JavaScript):

```
function cloneGraph(node) {  
  if (!node) return null;           // ఖాళీ graph → null  
  const map = new Map();             // original node → cloned node  
  
  const dfs = (curr) => {  
    // ఇప్పటికే clone చేసి ఉంటే, అదే clone ను వాడు (cycle ను ఆపే line)  
    if (map.has(curr)) return map.get(curr);  
  
    const copy = new Node(curr.val);  // కొత్త node (neighbors ఇంకా ఖాళీ)  
    map.set(curr, copy);              // ⚠️ neighbors నింపేముందే map లో పెట్టు!  
  
    for (const nb of curr.neighbors) { // ప్రతి neighbor ను (recursively) clone  
      copy.neighbors.push(dfs(nb));    // clone చేసి, copy యొక్క list కి కలుపు  
    }  
    return copy;  
  };  
  
  return dfs(node);  
}
```

Dry Run: nodes 1↔2 (రెండే nodes, ఒక edge)

```
dfs(1): map ఖాళీ → copy1=Node(1); map={1→copy1}  
  1.neighbors=[2] → dfs(2):  
    map లో 2 లేదు → copy2=Node(2); map={1→copy1, 2→copy2}  
    2.neighbors=[1] → dfs(1):  
      map లో 1 ఉంది! → return copy1 (★ ఇక్కడే infinite loop ఆగింది)
```

```
copy2.neighbors=[copy1]; return copy2
copy1.neighbors=[copy2]; return copy1
Result: copy1 ↔ copy2 – original తో identical structure, పూర్తిగా కొత్త objects. ✔
```

Complexity:

- **Time:** $O(V + E)$ – ప్రతి node ఒకసారి clone (map వల్ల), ప్రతి edge ఒకసారి traverse (neighbors loop).
Undirected కాబట్టి ప్రతి edge రెండువైపులా కనిపిస్తుంది → $O(2E) = O(E)$.
- **Space:** $O(V)$ – map లో V entries + recursion stack depth $O(V)$.

గుర్తుంచుకోవాల్సినది:

"Graph/tree structure తో objects ను clone / deep-copy చెయ్యాలంటే → **Map<original, copy>** ." ఇదే idea:
Copy List with Random Pointer (#138 – linked list version), any "serialize/deserialize with shared references." కీలక rule: **node ను create చేసిన వెంటనే map లో పెట్టు, తర్వాత children/neighbors నింపు** – లేకపోతే cycles infinite loop. ఇది "visited" ను "already-cloned" గా వాడే elegant పద్ధతి.

సాధారణ తప్పులు:

- **Map set ను అలస్యం చేయడం:** neighbors అన్న process చేశాక map.set చేస్తే, cycle మళ్ళీ అదే node కి వచ్చినప్పుడు map లో దొరకదు → infinite recursion. **Create → set → then recurse** క్రమం పాటించు.
- **Val ను key గా వాడటం:** ఈ problem లో val unique కాబట్టి పని చేస్తుంది, కానీ అలవాటుగా **node reference** ను key గా వాడు (Map object keys ను support చేస్తుంది). Duplicate వాల్క్యాలు ఉన్న graphs కి val-key తప్పు.
- **Shallow copy:** `copy.neighbors = curr.neighbors` అని రాస్తే – అది original nodes ను reference చేస్తుంది (deep copy కాదు). ప్రతి neighbor ను కూడా clone చేసి push చేయాలి.
- **null / single node:** input null అయితే null return. ఒక్క node, neighbors ఖాళీ అయినా code సరిగ్గా పని చేస్తుంది (loop skip).

4. Evaluate Division (LeetCode #399) – Medium

సమస్య: `equations[i] = [Ai, Bi]` మరియు `values[i]` ఇస్తారు – దీని అర్థం $A_i / B_i = values[i]$. ఇచ్చిన `queries[j] = [Cj, Dj]` ప్రతిదానికీ C_j / D_j విలువ లెక్కించాలి. లెక్కించలేకపోతే (variables తెలియకపోతే లేదా connect అవ్వకపోతే) **-1.0** return చెయ్యి.

Constraints: $1 \leq equations.length \leq 20$; $values > 0$; variables lowercase strings; queries లో unknown variables ఉండొచ్చు.

ఉదాహరణ:

```
equations = [["a","b"],["b","c"]], values = [2.0, 3.0]
అంటే: a/b = 2, b/c = 3
queries = [["a","c"],["b","a"],["a","e"],["a","a"],["x","x"]]
Output = [6.0, 0.5, -1.0, 1.0, -1.0]
a/c = (a/b)*(b/c) = 2*3 = 6
b/a = 1/(a/b) = 0.5
a/e = e తెలియదు → -1
```

$a/a = 1$ (a తెలుసు)
 $x/x = x$ తెలియదు $\rightarrow -1$

ఎలా ఆలోచించాలి:

ఇక్కడ "graph" అనే మాట ఎక్కడా లేదు — కానీ ఇది classic graph problem! ప్రతి variable ను ఒక **node** గా చూడు. $a/b = 2$ అంటే — a నుండి b కి ఒక **edge, weight 2** (a నుండి b కి వెళ్లే విలువ 2 తో గుణించు). అలాగే b నుండి a కి edge, weight $1/2$ (reverse direction లో reciprocal). ఇది **weighted directed graph**.

ఇప్పుడు a/c అంటే ఏమిటి? a నుండి c కి graph లో ఒక **path** వెతికి, ఆ path లోని edge weights అన్నీ **గుణిస్తే** వచ్చే విలువ! $a \rightarrow b$ ($\times 2$) $\rightarrow c$ ($\times 3$) = 6. ఎందుకంటే $a/c = (a/b) \times (b/c)$ — telescoping (మధ్యలో b లు cancel). Path లేకపోతే (a $\rightarrow$ e connect కాదు) $\rightarrow -1$.

కాబట్టి: build weighted graph, ప్రతి query కి DFS/BFS తో path వెతికి weights గుణించు. Query లోని ఏ variable graph లో లేకపోతే వెంటనే -1 . Src === dst అయి, ఆ variable graph లో ఉంటే $\rightarrow 1$ (ఏ node నుండి తనకే వెళ్ళడం = 1).

Optimal Approach:

Insight: Variables = nodes, $A/B = k$ = directed edge $A \rightarrow B$ (weight k) + $B \rightarrow A$ (weight $1/k$). Query C/D = product of edge weights on any path $C \rightarrow D$ (DFS).

Plan:

- graph = `Map<node, Map<neighbor, weight>>`. ప్రతి equation కి రెండు edges (forward k, backward $1/k$).
- ప్రతి query `[c, d]` : c లేదా d graph లో లేకపోతే -1 . `dfs(c, d, visited=Set)` — $c===d$ అయితే 1; లేకపోతే c యొక్క neighbors లోకి weight $\times$ sub దారి.
- Path లేకపోతే -1 .

Solution (JavaScript):

```
function calcEquation(equations, values, queries) {
  // graph: node -> Map(neighbor -> weight). a/b=k => a->b (k), b->a (1/k)
  const graph = new Map();
  for (let i = 0; i < equations.length; i++) {
    const [a, b] = equations[i], k = values[i];
    if (!graph.has(a)) graph.set(a, new Map());
    if (!graph.has(b)) graph.set(b, new Map());
    graph.get(a).set(b, k); // a/b = k
    graph.get(b).set(a, 1 / k); // b/a = 1/k
  }

  // src నుండి dst కి path weights ను గుణిస్తూ వెళ్ళు. path లేకపోతే -1.
  const dfs = (src, dst, visited) => {
    if (!graph.has(src) || !graph.has(dst)) return -1; // తెలియని variable
    if (src === dst) return 1; // తనకే: x/x = 1
    visited.add(src);
    for (const [next, w] of graph.get(src)) {
      if (visited.has(next)) continue; // cycle ను ఆపు
      const sub = dfs(next, dst, visited); // next నుండి dst కి
      if (sub !== -1) return w * sub; // దారి దొరికింది -> గుణించు
    }
    return -1; // ఏ neighbor ద్వారానూ చేరలేదు
  };
}
```

```
};

// ప్రతి query కి fresh visited set తో DFS
return queries.map(([c, d]) => dfs(c, d, new Set()));
}
```

Dry Run: a/b=2, b/c=3 ; query ["a","c"]

```
graph: a->{b:2}, b->{a:0.5, c:3}, c->{b:0.333}
dfs("a","c",{}):
  a≠c; visited={a}; neighbors of a: b
  dfs("b","c",{a}):
    b≠c; visited={a,b}; neighbors of b: a(visited→skip), c
    dfs("c","c",{a,b}): src===dst → return 1
    sub=1 (from c) → return 3 * 1 = 3
  sub=3 → return 2 * 3 = 6 ✓
query ["a","e"]: dfs → graph.has("e") false → -1 ✓
query ["a","a"]: graph.has("a") true, src===dst → 1 ✓
```

Complexity:

- **Time:** $O(Q \times (V + E))$ — ప్రతి query కి worst case మొత్తం graph traverse. Q = queries, V = distinct variables, E = equations×2.
- **Space:** $O(V + E)$ graph కి + $O(V)$ visited/recursion per query.
- **Optimization గమనిక:** ఒకే graph మీద చాలా queries ఉంటే **Floyd-Warshall** (అన్ని pairs precompute, $O(V^3)$) లేదా **Union-Find with weights** వాడొచ్చు. కానీ ఈ constraints కి (చిన్న graph) DFS per query సరిపోతుంది, code శుభ్రం.

గుర్తుంచుకోవాల్సినది:

"Elements మధ్య ratio / relationship / conversion ఇస్తే → **weighted graph, path product/sum.**" ఈ pattern: currency conversion (USD→EUR→INR rates గుణించు), unit conversion, "a is 2× b, b is 3× c." కీలక అనువాదం: **ratio = edge weight, reverse = reciprocal, query = path product.** "Graph" అని problem చెప్పకపోయినా graph గా చూడగలగడమే SSE-level skill.

సాధారణ తప్పులు:

- **Reverse edge మర్చిపోవడం:** a/b=k కి b→a = 1/k కూడా add చేయకపోతే, b/a queries fail. Undirected-like symmetry (reciprocal తో) కావాలి.
- **Unknown variable check మర్చిపోవడం:** query లో graph లో లేని variable (e, x) ఉంటే -1. graph.has check మొదట్లోనే. x/x కూడా x తెలియకపోతే -1 (కేవలం src===dst చూసి 1 ఇస్తే తప్పు).
- **Visited తో cycle handle చేయకపోవడం:** graph లో cycles ఉంటాయి (a↔b). Visited set లేకపోతే infinite recursion.
- **Fresh visited per query:** ప్రతి query కి కొత్త Set. ఒకే set ను reuse చేస్తే తర్వాతి queries తప్పు.

Pattern: Topological Sort

వివరణ

Topological Sort అంటే – ఒక **directed graph** లోని nodes ను ఒక **linear order** (వరుస) లో పేర్పడం, అలా పేర్పాలి: ప్రతి edge $u \rightarrow v$ కి, u ఎప్పుడూ v కంటే ముందు ఉండాలి. అంటే "u ముందు జరగాలి, తర్వాతే v" అనే dependency ని గౌరవించే వరుస.

ఇది **DAG (Directed Acyclic Graph)** కి మాత్రమే సాధ్యం. **Cycle ఉంటే topological order అసాధ్యం** – ఎందుకంటే $a \rightarrow b \rightarrow c \rightarrow a$ అయితే, a ముందా b ముందా? ఎవరూ ముందు కాలేరు (deadlock). కాబట్టి topological sort రెండు పనులు చేస్తుంది: (1) valid order ఇస్తుంది, లేదా (2) **cycle ను detect** చేస్తుంది.

రెండు standard methods ఉన్నాయి – రెండూ తెలుసుకో, interview లో ఏదైనా అడగొచ్చు:

Method 1 – Kahn's Algorithm (BFS + indegree):

- Indegree** = ఒక node లోకి వచ్చే edges సంఖ్య ("దీనికి ఎన్ని prerequisites ఉన్నాయి"). Indegree 0 = ఏ dependency లేదు, ఇప్పుడే చేయొచ్చు.
- Indegree 0 ఉన్న అన్ని nodes ను queue లో పెట్టు. ఒక్కొక్కటి తీసి order లో పెడుతూ, దాని neighbors అందరి indegree ను 1 తగ్గించు (ఒక prerequisite పూర్తయింది). ఏ neighbor indegree 0 అయితే queue లో పెట్టు.
- చివర్లో **order లో అన్ని nodes రాకపోతే** → **cycle ఉంది** (కొన్ని nodes ఎప్పటికీ indegree 0 కాలేదు).

Method 2 – DFS + cycle detection (3-color):

- ప్రతి node కి 3 states: **white** (చూడలేదు), **gray** (ప్రస్తుత DFS path లో ఉంది), **black** (పూర్తయింది).
- DFS లో ఒక **gray** node ను మళ్ళీ కలిస్తే → **back edge** → **cycle!**
- Node పూర్తయ్యాక (అన్ని neighbors చూశాక) దాన్ని ఒక stack/list లో push. చివర్లో ఆ list ను **reverse** చేస్తే topological order (post-order reversed).

Real-life Scenario

ఉదయం తయారవడం. చొక్కా వేసుకునే ముందు banian, షూస్ వేసుకునే ముందు socks, coat వేసుకునే ముందు చొక్కా – ఇవి dependencies. Topological sort = ఈ నియమాలన్నీ పాటించే ఒక valid dressing order. చాలా valid orders ఉండొచ్చు (socks ముందా watch ముందా – ఏదైనా సరే), కానీ ఏదీ ఒక rule ని violate చేయకూడదు.

Cycle = అసాధ్యమైన నియమాలు. "A కి ముందు B చేయాలి, B కి ముందు C, C కి ముందు A" – ఇది చేయడం అసాధ్యం (deadlock). University లో "course X కి prerequisite Y, Y కి prerequisite X" అంటే ఏ course తో మొదలుపెట్టాలో తెలియదు – degree పూర్తవదు. Topological sort ఈ deadlock ను గుర్తిస్తుంది.

ఎలా గుర్తించాలి (Recognition Signals)

- "Order / sequence / schedule" + "before / prerequisite / dependency / must come first" → topological sort.
- "Can you finish / is it possible" + **directed dependencies** → cycle detection (topo sort variant).

- **Build order, task scheduling, course schedule, compilation order, package dependency resolution** – అన్నీ topo sort.
- **Kahn (BFS) ఎప్పుడు:** order కావాలి + cycle detect + iterative (stack overflow భయం లేదు). ఇది default, సులభం.
- **DFS ఎప్పుడు:** recursion comfortable, లేదా problem కి post-order సహజంగా సరిపోతే.

Kahn's Algorithm template (BFS + indegree)

```
// n nodes (0..n-1), edges = [[u, v], ...] అంటే u ముందు, తర్వాత v (u → v)
function topoSortKahn(n, edges) {
  const graph = Array.from({ length: n }, () => []);
  const indegree = new Array(n).fill(0);
  for (const [u, v] of edges) {
    graph[u].push(v); // u → v
    indegree[v]++;    // v కి ఒక prerequisite (u) పెరిగింది
  }

  // dependency లేని (indegree 0) nodes తో మొదలుపెట్టు
  const queue = [];
  for (let i = 0; i < n; i++) if (indegree[i] === 0) queue.push(i);

  const order = [];
  while (queue.length) {
    const node = queue.shift(); // ఇప్పుడు చేయదగినది
    order.push(node);
    for (const next of graph[node]) {
      indegree[next]--; // node పూర్తయింది → next కి ఒక prereq తగ్గింది
      if (indegree[next] === 0) queue.push(next); // dependencies అయిపోయాయి
    }
  }

  // అన్ని nodes order లోకి వస్తే valid; రాకపోతే cycle ఉంది
  return order.length === n ? order : []; // [] = cycle (order అసాధ్యం)
}
```

DFS + cycle detection template

```
function topoSortDFS(n, edges) {
  const graph = Array.from({ length: n }, () => []);
  for (const [u, v] of edges) graph[u].push(v);

  const state = new Array(n).fill(0); // 0=white(unseen) 1=gray(in-path) 2=black(done)
  const order = [];
  let hasCycle = false;

  const dfs = (node) => {
    state[node] = 1; // gray: ప్రస్తుత path లోకి enter
    for (const next of graph[node]) {
      if (state[next] === 1) { hasCycle = true; return; } // gray ను మళ్ళీ కలిశాం → cycle!
      if (state[next] === 0) dfs(next); // white → లోతుకి వెళ్ళు
      // black అయితే skip (అప్పటికే పూర్తిగా process అయింది)
    }
  }
}
```

```

state[node] = 2; // black: పూర్తయింది
order.push(node); // post-order లో push
};

for (let i = 0; i < n; i++) if (state[i] === 0) dfs(i);
if (hasCycle) return [];
return order.reverse(); // ⚠️ post-order reversed = topological order
}

```

Complexity

- **Time:** $O(V + E)$ — రెండు methods లోనూ ప్రతి node ఒకసారి, ప్రతి edge ఒకసారి.
- **Space:** $O(V + E)$ — adjacency list + indegree/state arrays + queue/recursion.

గుర్తుంచుకో: Kahn లో "processed nodes < total" అంటే cycle. DFS లో "gray node ను మళ్ళీ కలవడం" అంటే cycle. ఈ రెండు cycle-detection signals SSE interview లో మాటిమాటికీ అడుగుతారు.

5. Course Schedule (LeetCode #207) — Medium

సమస్య: numCourses courses ఉన్నాయి (0 నుండి numCourses-1). prerequisites[i] = [a, b] అంటే — course a తీసుకోవాలంటే ముందు course b పూర్తి చేయాలి. అన్ని courses పూర్తి చేయడం సాధ్యమేనా (true / false) చెప్పాలి.

Constraints: $1 \leq \text{numCourses} \leq 2000$; $0 \leq \text{prerequisites.length} \leq 5000$; jodi లు unique.

ఉదాహరణ:

```

numCourses = 2, prerequisites = [[1,0]]
→ course 1 కి course 0 కావాలి. 0 → 1 order సాధ్యం. Output: true

numCourses = 2, prerequisites = [[1,0],[0,1]]
→ 1 కి 0 కావాలి, 0 కి 1 కావాలి → deadlock (cycle). Output: false

```

ఎలా ఆలోచించాలి:

"Course a కి course b prerequisite" అంటే — dependency. దీన్ని **directed graph** గా చూడు: courses = nodes. [a, b] (a కి b కావాలి) అంటే edge **b → a** (b ముందు, తర్వాత a). ఇప్పుడు "అన్ని courses పూర్తి చేయగలమా?" అనే ప్రశ్న ఏమవుతుంది?

Dependencies ను ఒక valid order లో పేర్చగలిగితే (topological order) → అన్నీ పూర్తి చేయొచ్చు. పేర్చలేకపోతే — అంటే graph లో **cycle** ఉంటే — పూర్తి చేయలేం (a కి b కావాలి, b కి a కావాలి → ఎవరూ ముందు కాలేరు). కాబట్టి ఈ problem నిజానికీ: **"ఈ directed graph లో cycle ఉందా?"** Cycle లేకపోతే true, ఉంటే false.

Cycle detection కి Kahn's algorithm (BFS + indegree) చాలా శుభం: indegree 0 ఉన్న courses (prerequisites లేనివి) ముందు తీసుకో, వాటిని పూర్తి చేస్తూ dependents ని unlock చెయ్యి. చివర్లో **అన్ని courses process అయితే** cycle లేదు → true. కొన్ని మిగిలిపోతే (ఎప్పటికీ indegree 0 కానివి) → cycle → false.

Optimal Approach:

Insight: "అన్నీ finish చేయగలమా?" = "dependency graph acyclic ఆ?" = Kahn's topological sort లో processed count === numCourses ఆ?

Plan:

1. graph ($b \rightarrow a$ edges) + indegree array build. $[a, b] \rightarrow \text{graph}[b].\text{push}(a)$, $\text{indegree}[a]++$.
2. indegree 0 ఉన్న courses queue లోకి.
3. BFS: node తీసి $\text{completed}++$; ప్రతి dependent indegree-- ; 0 అయితే queue.
4. $\text{completed} === \text{numCourses}$ ఆ return.

Solution (JavaScript):

```
function canFinish(numCourses, prerequisites) {
  // graph: prereq → ఆ prereq మీద ఆధారపడే courses. indegree: ఒక్కో course prereq count
  const graph = Array.from({ length: numCourses }, () => []);
  const indegree = new Array(numCourses).fill(0);

  for (const [course, prereq] of prerequisites) {
    graph[prereq].push(course); // prereq ముందు → course తర్వాత (prereq → course)
    indegree[course]++;        // course కి ఒక prerequisite పెరిగింది
  }

  // prerequisites లోని (indegree 0) courses ను ఇప్పుడే తీసుకోవచ్చు
  const queue = [];
  for (let i = 0; i < numCourses; i++) if (indegree[i] === 0) queue.push(i);

  let completed = 0;
  while (queue.length) {
    const course = queue.shift(); // ఈ course పూర్తి చేశాం
    completed++;
    for (const next of graph[course]) {
      indegree[next]--; // దీని ఒక prerequisite (course) పూర్తయింది
      if (indegree[next] === 0) queue.push(next); // అన్ని prereqs అయ్యాయి → unlock
    }
  }

  // అన్ని courses పూర్తయితే cycle లేదు; లేకపోతే cycle వల్ల deadlock
  return completed === numCourses;
}
```

Dry Run: numCourses=4, prerequisites=[[1,0],[2,0],[3,1],[3,2]]

```
graph: 0→[1,2], 1→[3], 2→[3], 3→[]
indegree: [0,1,1,2] (course0:0, course1:1, course2:1, course3:2)
queue (indegree 0): [0]
pop 0: completed=1; neighbors 1,2 → indegree[1]=0(push), [2]=0(push); queue=[1,2]
pop 1: completed=2; neighbor 3 → indegree[3]=1; queue=[2]
pop 2: completed=3; neighbor 3 → indegree[3]=0(push); queue=[3]
pop 3: completed=4; no neighbors; queue=[]
completed(4) === numCourses(4) → true ✓
```

```
cycle case [[1,0],[0,1]]: indegree=[1,1], ఏదీ 0 కాదు → queue ఖాళీ →  
completed=0 ≠ 2 → false ✓
```

Complexity:

- **Time:** $O(V + E)$ — $V = \text{numCourses}$, $E = \text{prerequisites}$. Graph build $O(E)$, BFS ప్రతి node/edge ఒకసారి.
- **Space:** $O(V + E)$ — adjacency list + indegree + queue.

గుర్తుంచుకోవాల్సినది:

"Can finish / is it possible + prerequisites/dependencies" = directed graph cycle detection = topological sort. కీలక realization: **అన్నీ పూర్తి చేయగలగడం** $\iff$ **cycle లేకపోవడం**. Kahn లో "processed count == total ఆ?" ఒక్కటే check. ఇదే idea: **Course Schedule II** (#210 — order కూడా కావాలి), **Alien Dictionary**, **Build order / task scheduling**, deadlock detection. Edge direction (prereq $\rightarrow$ course) ను జాగ్రత్తగా set చెయ్యి — తిరగేస్తే logic తప్పు.

సాధారణ తప్పులు:

- **Edge direction తిరగవేయడం:** $[a, b] = \text{"a కి b కావాలి"} = b \rightarrow a$. దీన్ని $a \rightarrow b$ గా పెడితే indegree logic మొత్తం తప్పు. "Prerequisite ముందు, dependent తర్వాత" గుర్తుంచుకో.
- **Push వద్ద indegree-0 check మర్చిపోవడం:** neighbor indegree ను తగ్గించాక **0 అయితేనే** queue లో పెట్టాలి. ప్రతిసారి పెడితే duplicates + తప్పు.
- **Disconnected graph:** అన్ని courses connect అవ్వకపోవచ్చు. Indegree-0 initial scan **అన్ని** nodes మీద చెయ్యాలి (ఒకే starting point కాదు). Kahn దీన్ని సహజంగా handle చేస్తుంది.
- **Self-loop:** $[1,1]$ (course 1 కి course 1 prerequisite) — indegree[1] ఎప్పటికీ 0 కాదు $\rightarrow$ **false** (correct, self-dependency = deadlock).

6. Course Schedule II (LeetCode #210) — Medium

సమస్య: #207 లాగే — numCourses courses, $\text{prerequisites}[i] = [a, b]$ (a కి ముందు b). ఇప్పుడు కేవలం "సాధ్యమేనా" కాదు, **అన్ని courses తీసుకునే ఒక valid order** (topological order) ను array గా return చెయ్యాలి. అలాంటి order సాధ్యం కాకపోతే (cycle) **ఖాళీ array []** return. చాలా valid orders ఉంటే ఏదైనా ఒకటి చాలు.

Constraints: $1 \leq \text{numCourses} \leq 2000$; $0 \leq \text{prerequisites.length} \leq \text{numCourses} * (\text{numCourses} - 1)$.

ఉదాహరణ:

```
numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]  
Output: [0,1,2,3] (లేదా [0,2,1,3] — రెండూ valid)  
0 ముందు; తర్వాత 1,2 (ఏ order అయినా); చివర 3.  
  
numCourses = 2, prerequisites = [[0,1],[1,0]]  
Output: [] (cycle  $\rightarrow$  order అసాధ్యం)
```

ఎలా ఆలోచించాలి:

ఇది #207 కి direct extension. అక్కడ మనం "cycle ఉందా?" మాత్రమే చూశాం (count తో). ఇక్కడ అదనంగా **actual order** కావాలి. Kahn's algorithm లో మనం courses ను ఒక్కొక్కటిగా queue నుండి తీసేటప్పుడు – **ఆ order యే topological order!** ఎందుకంటే మనం ఎప్పుడూ "అన్ని prerequisites పూర్తయిన" course ను మాత్రమే తీస్తాం. కాబట్టి #207 code కి ఒక్క మార్పు: completed counter బదులు, తీసిన courses ను order array లో push చెయ్యి.

చివర్లో: order.length === numCourses అయితే అది valid answer. లేకపోతే (cycle వల్ల కొన్ని courses ఎప్పటికీ queue లోకి రాలేదు) → [] return. అంత సింపుల్. Kahn's algorithm order ను **ఉచితంగా** ఇస్తుంది – అదే దీని అందం.

Optimal Approach:

Insight: Kahn's BFS లో queue నుండి తీసే క్రమమే topological order. Count బదులు order array collect చెయ్యి; length === numCourses కాకపోతే cycle → [] .

Plan:

1. #207 లాగే graph + indegree.
2. indegree-0 courses queue లోకి.
3. BFS: తీసిన course ను order.push ; neighbors indegree-- , 0 అయితే queue.
4. order.length === numCourses ? order : [] .

Solution (JavaScript):

```
function findOrder(numCourses, prerequisites) {
    const graph = Array.from({ length: numCourses }, () => []);
    const indegree = new Array(numCourses).fill(0);

    for (const [course, prereq] of prerequisites) {
        graph[prereq].push(course); // prereq → course
        indegree[course]++;
    }

    const queue = [];
    for (let i = 0; i < numCourses; i++) if (indegree[i] === 0) queue.push(i);

    const order = []; // #207 లో ఇది కేవలం counter; ఇక్కడ actual order
    while (queue.length) {
        const course = queue.shift();
        order.push(course); // ఈ course ను ఇప్పుడు తీసుకోవచ్చు → order లో చేర్చు
        for (const next of graph[course]) {
            indegree[next]--;
            if (indegree[next] === 0) queue.push(next);
        }
    }

    // అన్ని order లోకి వస్తే valid; రాకపోతే cycle → ఖాళీ array
    return order.length === numCourses ? order : [];
}
```

Dry Run: numCourses=4, prerequisites=[[1,0],[2,0],[3,1],[3,2]]

```
graph: 0→[1,2], 1→[3], 2→[3], 3→[]
indegree: [0,1,1,2]
queue: [0]
pop 0 → order=[0]; 1→indeg0(push), 2→indeg0(push); queue=[1,2]
pop 1 → order=[0,1]; 3→indeg1; queue=[2]
pop 2 → order=[0,1,2]; 3→indeg0(push); queue=[3]
pop 3 → order=[0,1,2,3]; queue=[]
order.length(4)===4 → return [0,1,2,3] ✓
```

Complexity:

- **Time:** $O(V + E)$ — #207 తో identical.
- **Space:** $O(V + E)$ — graph + indegree + queue + order array.

గుర్తుంచుకోవాల్సినది:

Kahn's algorithm లో "queue నుండి dequeue చేసే క్రమమే" **topological order** — ప్రత్యేకంగా ఏమీ చేయనక్కర్లేదు, tap చేసి collect చేస్తే చాలు. #207 (yes/no) → #210 (order) మధ్య తేడా కేవలం "count vs collect." ఇదే idea: **Alien Dictionary** (అక్షరాల order deduce చెయ్యి), **Parallel Courses** (minimum semesters = topo levels), **Sort Items by Groups**. DFS variant కావాలంటే: post-order reversed. Interview లో "నాకు order కూడా కావాలి" అంటే panic వద్దు — Kahn ఇస్తుంది.

సాధారణ తప్పులు:

- **Cycle లో partial order return చేయడం:** length check మర్చిపోతే, cycle ఉన్నప్పుడు అసంపూర్ణ order return అవుతుంది. `order.length === numCourses` కాకపోతే తప్పుకుండా `[]` .
- **Order ను reverse చేయడం:** Kahn's లో order సరైన దిశలోనే ఉంటుంది (prerequisite ముందు). **DFS** approach లో మాత్రం post-order ను reverse చేయాలి — రెండింటినీ కలిపి confuse అవ్వకు.
- **Edge direction:** మళ్ళీ, `[a,b] = b → a` . తప్పు direction = తప్పు order.
- **Empty prerequisites:** prerequisites `[]` అయితే అన్ని indegree 0 → `[0,1,...,n-1]` valid order (ఏ order అయినా సరే).

7. Snakes and Ladders (LeetCode #909) — Medium

సమస్య: $n \times n$ board లో cells 1 నుండి n^2 వరకు **boustrophedon** (పాము దారి — కింది row ఎడమ→కుడి, తర్వాతి row కుడి→ఎడమ, alternate) క్రమంలో numbered. Cell 1 (కింది-ఎడమ) నుండి మొదలు, n^2 కి చేరాలి. ప్రతి move: dice వేసి 1..6 లో ఏదైనా సంఖ్య ఎంచుకుని అన్ని ముందుకీ. `board[r][c] != -1` అయితే అది **snake లేదా ladder** — ఆ cell కి వెళ్తే వెంటనే `board[r][c]` destination కి jump అవుతావు. n^2 కి చేరడానికి **కనీస moves** ఎన్ని? సాధ్యం కాకపోతే **-1** .

Constraints: $2 \leq n \leq 20$; `board[r][c]` **-1** లేదా $1..n^2$; start/end cells కి snake/ladder ఉండదు.

ఉదాహరణ:

`n=6` board (కొన్ని ladders/snakes తో): Output = 4

1 → (dice) → ... ladder ఎక్కి ... → 36. కనీసం 4 dice throws.

ఎలా ఆలోచించాలి:

"కనీసం moves" — ఈ మాట వినగానే alarm మోగాలి: **shortest path in an unweighted graph** → **BFS**. ఎందుకు unweighted? ప్రతి dice throw (1..6 ఏదైనా) **ఒకే move** — cost అన్నిటికీ సమానం (1). కాబట్టి levels = moves.

Graph ఎలా? ప్రతి cell $1..n^2$ ఒక **node**. Cell x నుండి edges: $x+1, x+2, \dots, x+6$ (dice ఆరు అవకాశాలు). కానీ ఆ target cell కి snake/ladder ఉంటే, నిజమైన destination ఆ jump చివర. అంటే edge $x \rightarrow (x+d$ కి snake/ladder ఉంటే దాని destination, లేకపోతే $x+d$).

ఒక చిక్కు: board 2D (r, c), కానీ cells 1D numbers (boustrophedon). కాబట్టి **cell number** → **(row, col)** మార్చే helper కావాలి. Number s కి: $quot = (s-1)/n$ (కింది నుండి ఎన్నో row), $row = n-1-quot$. Column: ఆ row సరి quotient అయితే ఎడమ→కుడి (rem), బేసి అయితే కుడి→ఎడమ ($n-1-rem$) — ఇదే పాము దారి (zigzag).

BFS: cell 1 నుండి, ప్రతి level లో 6 possible next cells explore, visited తో duplicate ఆపు, n^2 చేరినప్పుడు level = answer.

Optimal Approach:

Insight: Cells = nodes, dice 1..6 = edges, snake/ladder = edge redirect. Unweighted shortest path → BFS with levels = moves.

Plan:

1. `getRC(s)` — boustrophedon cell number → `[row, col]`.
2. BFS from cell 1: queue `[cell, moves]`, visited array size n^2+1 .
3. ప్రతి cell కి $d=1..6$: `next = cell+d` ($> n^2$ అయితే break); ఆ `next` cell కి `board[r][c] != -1` అయితే `next = board[r][c]`. Unvisited అయితే enqueue `[next, moves+1]`.
4. `cell === n^2` → return moves. Queue ఖాళీ అయితే `-1`.

Solution (JavaScript):

```
function snakesAndLadders(board) {
    const n = board.length;

    // boustrophedon cell number (1..n^2) → [row, col]
    const getRC = (s) => {
        const quot = Math.floor((s - 1) / n); // కింది నుండి ఎన్నో వరుస (0-indexed)
        const rem = (s - 1) % n;
        const row = n - 1 - quot; // board పైనుండి index (కింద = n-1)
        const col = quot % 2 === 0 ? rem : n - 1 - rem; // సరి row: ఎడమ→కుడి, బేసి: reverse
        return [row, col];
    };

    const visited = new Array(n * n + 1).fill(false);
    const queue = [[1, 0]]; // [cell, moves] — cell 1 నుండి 0 moves
    visited[1] = true;

    while (queue.length) {
        const [cell, moves] = queue.shift();
```

```

if (cell === n * n) return moves; // గమ్యం చేరాం - ఇదే shortest (BFS)
for (let d = 1; d <= 6; d++) { // dice 6 అవకాశాలు
  const next = cell + d;
  if (next > n * n) break; // board దాటితే ఆగు
  const [r, c] = getRC(next);
  const dest = board[r][c] === -1 ? next : board[r][c]; // snake/ladder jump
  if (!visited[dest]) {
    visited[dest] = true; // ⚠ enqueue వద్దే mark (dequeue కాదు)
    queue.push([dest, moves + 1]);
  }
}
}
return -1; // ఎప్పటికీ చేరలేదు
}

```

Dry Run: small $n=2$, board= $\begin{bmatrix} -1 & -1 \\ -1 & 3 \end{bmatrix}$ (cell 1 నుండి 4 కి)

```

cells: (1,0)=1, (1,1)=2, (0,1)=3, (0,0)=4
      (getRC: 1→(1,0), 2→(1,1), 3→(0,1), 4→(0,0))
board[0][1]=3 → cell 3 కి వెళ్తే ladder → cell 3 (ఇక్కడ same, demo).
BFS: queue=[[1,0]], visited={1}
pop [1,0]: cell1≠4. d=1→next2, getRC(2)=(1,1),board=-1→dest2, push[2,1];
          d=2→next3,(0,1),board[0][1]=3→dest3, push[3,1];
          d=3→next4=? n²=4 not > 4 → getRC(4)=(0,0),board=-1→dest4,push[4,1]
pop [2,1]: ≠4 ... (already visited neighbors)
pop [3,1]: ≠4 ...
pop [4,1]: cell4===n²(4) → return 1 ✓ (ఒక్క dice throw తో 1→4)

```

Complexity:

- **Time:** $O(n^2)$ — cells n^2 nodes, ప్రతి cell కి ≤ 6 edges → $O(6 \cdot n^2) = O(n^2)$. ప్రతి cell ఒకసారి visit.
- **Space:** $O(n^2)$ — visited array + queue (worst case అన్ని cells).

గుర్తుంచుకోవాల్సినది:

"Minimum moves / fewest steps in a grid/board where each move has equal cost = BFS." Levels = distance. కీలక skill: board/state ను **implicit graph** గా చూడటం (cell = node, allowed move = edge). ఇదే idea: **Open the Lock** (#752 — dial states), **Jump Game III**, **Minimum Knight Moves** (chess board BFS), **Sliding Puzzle**. ఇక్కడ ప్రత్యేక సవాలు — 1D↔2D coordinate mapping (boustrophedon); దాన్ని ఒక clean helper లో వేరు చేయడం bug-free code కి కీలకం.

సాధారణ తప్పులు:

- **Boustrophedon mapping తప్పు:** row parity (సరి/బేసి) బట్టి column direction flip మర్చిపోతే, snake దారి తప్పు → wrong answer. getRC ను చిన్న board మీద verify చెయ్యి.
- **Dequeue వద్ద visited mark చేయడం:** BFS లో **enqueue వద్దే** mark చెయ్యాలి; లేకపోతే ఒక cell multiple సార్లు queue లో పడి TLE/wrong.
- **Snake/ladder loop:** ఒక ladder ఇంకో snake కి దారితీయొచ్చు; కానీ మనం **ఒకే jump** (target cell value) మాత్రమే follow చేస్తాం (chained jumps కాదు) — problem definition అదే. Visited ఇది సురక్షితంగా handle చేస్తుంది.

- **next > n² break vs continue:** dice increasing కాబట్టి **next > n²** వస్తే మిగతా d లూ కూడా దాటుతాయి → **break** (కొంచెం optimize). **continue** కూడా correct, కానీ break శుభ్రం.

8. Minimum Genetic Mutation (LeetCode #433) — Medium

సమస్య: ఒక gene string 8 అక్షరాలు, ప్రతి అక్షరం 'A', 'C', 'G', 'T' లలో ఒకటి. startGene నుండి endGene కి చేరాలి. ఒక **mutation** = ఏదైనా ఒక్క position లో అక్షరం మార్చడం. కానీ ప్రతి **intermediate gene (మరియు endGene)** bank లో ఉండాలి (valid genes list). startGene నుండి endGene కి చేరడానికి **కనీస mutations** ఎన్ని? సాధ్యం కాకపోతే **-1**.

Constraints: gene length 8; **0** <= bank.length <= **10**; అన్ని genes **A/C/G/T** మాత్రమే.

ఉదాహరణ:

```
startGene="AACCGGTT", endGene="AACCGGTA", bank=["AACCGGTA"] → 1
startGene="AACCGGTT", endGene="AAACGGTA", bank=["AACCGGTA","AACCGCTA","AAACGGTA"] → 2
AACCGGTT → AACCGGTA → AAACGGTA (2 mutations)
```

ఎలా ఆలోచించాలి:

మళ్ళీ "కనీస mutations / minimum steps" — **BFS shortest path**. ఇక్కడ graph చాలా abstract: ప్రతి valid gene ఒక **node**. రెండు genes మధ్య **edge** ఉంది — ఒకే ఒక్క position తేడా ఉంటే (one-character mutation) **మరియు** ఆ target gene bank లో ఉంటే. "startGene నుండి endGene కి కనీస edges" = BFS levels.

ఇక్కడ నేర్చుకోవాల్సిన కీలక skill: **neighbors ను explicit గా ఇవ్వరు — వాటిని generate చేయాలి**. ఒక gene యొక్క neighbors ఏవి? ప్రతి position (8) లో, ప్రస్తుత అక్షరం కాకుండా మిగతా 3 అక్షరాలతో మార్చి, ఆ కొత్త gene bank లో ఉందా చూడు. ఉంటే అది valid neighbor. అంటే ఒక్కో gene కి **8 × 3 = 24** candidate mutations, వాటిలో bank లో ఉన్నవే edges.

BFS: startGene నుండి, level by level, valid unvisited mutations explore. endGene చేరిన level = answer. Bank ను **Set** లోకి మార్చు (O(1) lookup). Visited తో genes ను మళ్ళీ process చేయకుండా ఆపు.

Optimal Approach:

Insight: Genes = nodes; edge = one-char mutation that lands in bank. Neighbors generate చెయ్యి (8 positions × 3 letters), bank లో ఉన్నవే valid. BFS levels = mutations.

Plan:

1. bankSet = new Set(bank) . startGene === endGene → 0. endGene bank లో లేకపోతే → **-1**.
2. BFS: queue [gene, steps] , visited set.
3. ప్రతి gene కి 8 positions × {A,C,G,T} (same char skip) mutations; bank లో & unvisited అయితే enqueue steps+1.
4. endGene దొరికితే steps return; queue ఖాళీ → **-1**.

Solution (JavaScript):

```

function minMutation(startGene, endGene, bank) {
  if (startGene === endGene) return 0; // ఇప్పటికే చేరాం
  const bankSet = new Set(bank);
  if (!bankSet.has(endGene)) return -1; // endGene valid కాదు → చేరలేం
  const letters = ['A', 'C', 'G', 'T'];

  const visited = new Set([startGene]);
  const queue = [[startGene, 0]]; // [gene, mutations so far]

  while (queue.length) {
    const [gene, steps] = queue.shift();
    if (gene === endGene) return steps; // BFS → మొదటిసారి చేరిందే shortest

    // neighbors ను generate చెయ్యి: ప్రతి position × ప్రతి letter
    for (let i = 0; i < gene.length; i++) {
      for (const ch of letters) {
        if (ch === gene[i]) continue; // అదే అక్షరం → mutation కాదు
        const mutated = gene.slice(0, i) + ch + gene.slice(i + 1);
        // valid neighbor = bank లో ఉంది + ఇంతకుముందు చూడలేదు
        if (bankSet.has(mutated) && !visited.has(mutated)) {
          visited.add(mutated); // enqueue వద్దే mark
          queue.push([mutated, steps + 1]);
        }
      }
    }
  }
  return -1; // ఏ దారి endGene కి చేరలేదు
}

```

Dry Run: start="AACCGGTT", end="AAACGGTA", bank=["AACCGGTA","AACCGCTA","AAACGGTA"]

```

bankSet has end("AAACGGTA") ✓; start≠end
queue=[["AACCGGTT",0]], visited={AACCGGTT}
pop [AACCGGTT,0]: ≠end. mutations → "AACCGGTA" (bank✓) enqueue [.,1] visited+=
pop [AACCGGTA,1]: ≠end. mutations → "AACCGCTA"(bank✓)[.,2], "AAACGGTA"(bank✓)[.,2]
pop [AACCGCTA,2]: ≠end ...
pop [AAACGGTA,2]: ===end → return 2 ✓

```

Complexity:

- **Time:** $O(B \times L \times 4) \approx O(B \cdot L)$ — B = bank size, L = gene length (8). ప్రతి gene కి $L \times 4$ neighbors generate, ప్రతి slice $O(L)$. ఇక్కడ చిన్న constants (L=8, B≤10) కాబట్టి చాలా వేగం.
- **Space:** $O(B)$ — visited + queue (genes bank subset).

గుర్తుంచుకోవాల్సినది:

"కనీస transformations, ఒక్కో step లో ఒక చిన్న మార్పు, valid set లో ఉండాలి = BFS on implicit word/state graph." ఇది **Word Ladder** (#127 — తర్వాతి problem) కి కవల సోదరుడు — అక్కడ 26 letters, ఇక్కడ 4. Neighbors ను generate చేయడం (అన్ని single-char మార్పులు, valid set filter) ఇక్కడి కీలక technique. ఇదే idea: Open the Lock, word transformation puzzles. State graph implicit అయినప్పుడు BFS + "neighbor generator" + visited — ఈ combo గుర్తుంచుకో.

సాధారణ తప్పులు:

- **endGene bank లో ఉందా చూడకపోవడం:** endGene bank లో లేకపోతే ఎప్పటికీ చేరలేం. ముందే **-1** (లేకపోతే BFS వృథాగా తిరుగుతుంది – తప్పు కాదు కానీ inefficient; అయితే start===end edge case కి ముందు 0 check అవసరం).
- **start === end edge case:** ముందు 0 return చేయకపోతే, endGene bank లో లేకపోతే check **-1** ఇచ్చేస్తుంది (తప్పు – mutations అవసరం లేదు). కాబట్టి start===end → **0** ను bank check **కంటే ముందు** పెట్టు.
- **Same-char mutation:** ch === gene[i] skip చేయకపోతే, mutation కాని "మార్పు" ను లెక్కించి visited/queue ను వృథా చేస్తుంది (correctness కి పెద్ద harm లేదు కానీ అనవసరం).
- **Visited మర్చిపోవడం:** valid mutations graph లో cycles ఉంటాయి (A→B అయితే B→A కూడా). Visited లేకపోతే infinite / TLE.

9. Word Ladder (LeetCode #127) – Hard

సమస్య: beginWord , endWord , మరియు wordList ఇస్తారు. beginWord నుండి endWord కి ఒక **transformation sequence** – ప్రతి step లో **ఒక్క అక్షరం మార్పు**, ప్రతి intermediate word (endWord తో సహా) wordList లో ఉండాలి. అలాంటి sequence లో **ఎన్ని words ఉంటాయో** (అతి చిన్న sequence పొడవు, రెండు ends తో కలిపి) return చెయ్యి. సాధ్యం కాకపోతే **0** .

Constraints: అన్ని words ఒకే length; lowercase; **beginWord != endWord** ; wordList లో duplicates లేవు. beginWord wordList లో ఉండనవసరం లేదు.

ఉదాహరణ:

```
beginWord="hit", endWord="cog", wordList=["hot","dot","dog","lot","log","cog"]
Output: 5 → hit → hot → dot → dog → cog (5 words)

endWord="cog", wordList=["hot","dot","dog","lot","log"] (cog లేదు)
Output: 0 (endWord wordList లో లేదు → చేరలేం)
```

ఎలా ఆలోచించాలి:

ఇది Minimum Genetic Mutation (#433) యొక్క పెద్ద version – genes బదులు words, 4 letters బదులు 26 letters (a-z). మళ్ళీ **BFS shortest path**. ప్రతి word ఒక **node**. రెండు words మధ్య **edge** – ఒకే ఒక్క అక్షరం తేడా (ఒక valid transformation). "beginWord నుండి endWord కి కనీస transformations." Answer = words count = edges + 1 = BFS levels.

మళ్ళీ neighbors ను generate చేయాలి: ఒక word కి, ప్రతి position లో a-z 26 అక్షరాలతో మార్చి, ఆ candidate wordList లో ఉందా చూడు. ఒక్కో word కి $L \times 26$ candidates (L = word length). Level counting జాగ్రత్త: beginWord ఒంటరిగా level 1 (sequence లో మొదటి word). ప్రతి BFS layer level ను 1 పెంచుతుంది.

Level ను ఎలా లెక్కించాలి? రెండు పద్ధతులు: (a) queue లో **[word, level]** పెట్టు, లేదా (b) **level-by-level BFS** – ప్రస్తుత layer మొత్తం process చేసి level++ చెయ్యి. (b) శుభ్రం, తర్వాత bidirectional BFS కి extend చేయడం సులభం.

Brute Force / naive:

ప్రతి jodi words మధ్య "one char apart ఆ?" చూసి explicit adjacency graph build చేయడం $\rightarrow O(N^2 \times L)$ (N words, ప్రతి jodi కి L comparison). N పెద్దది అయితే నెమ్మది. దీనికి బదులు neighbors ను on-the-fly generate చేస్తే ($L \times 26$ per word) graph ను ముందే build చేయనవసరం లేదు.

Optimal Approach:

Insight: Words = nodes; edge = one-letter difference within wordList. BFS levels = sequence length. Neighbors ను generate చెయ్యి (L positions $\times$ 26 letters), wordSet filter.

Plan:

1. wordSet = new Set(wordList) . endWord లేకపోతే $\rightarrow 0$.
2. Level-by-level BFS from beginWord , level = 1 .
3. ప్రతి layer: word === endWord $\rightarrow$ level. లేకపోతే ప్రతి position $\times$ a-z candidate; wordSet లో & unvisited అయితే next layer కి, visited mark.
4. Layers అయ్యాక level++. Queue ఖాళీ $\rightarrow 0$.

Solution (JavaScript):

```
function ladderLength(beginWord, endWord, wordList) {
  const wordSet = new Set(wordList);
  if (!wordSet.has(endWord)) return 0; // endWord చేరలేని target
  const a = 'a'.charCodeAt(0);

  let queue = [beginWord]; // ప్రస్తుత BFS layer
  const visited = new Set([beginWord]);
  let level = 1; // beginWord ఒక్కటే  $\rightarrow$  sequence length 1

  while (queue.length) {
    const nextLayer = [];
    for (const word of queue) {
      if (word === endWord) return level; // చేరాం - ఇదే shortest length
      // ఈ word యొక్క neighbors: ప్రతి position  $\times$  26 letters
      for (let i = 0; i < word.length; i++) {
        for (let k = 0; k < 26; k++) {
          const ch = String.fromCharCode(a + k);
          if (ch === word[i]) continue; // అదే అక్షరం  $\rightarrow$  మార్పు కాదు
          const cand = word.slice(0, i) + ch + word.slice(i + 1);
          if (wordSet.has(cand) && !visited.has(cand)) {
            visited.add(cand); // enqueue వద్దే mark
            nextLayer.push(cand);
          }
        }
      }
    }
    queue = nextLayer; // తర్వాతి ring కి move
    level++; // ఒక layer దాటాం  $\rightarrow$  distance +1
  }
  return 0; // ఏ దారి endWord కి చేర్చలేదు
}
```

Dry Run: begin="hit", end="cog", list=["hot","dot","dog","lot","log","cog"]

```
wordSet has "cog" ✓. queue=["hit"], visited={hit}, level=1
Layer level=1: "hit"≠cog. neighbors in set: "hot". nextLayer=[hot]. level=2
Layer level=2: "hot"≠cog. neighbors: "dot","lot". nextLayer=[dot,lot]. level=3
Layer level=3: "dot"→"dog"; "lot"→"log". nextLayer=[dog,log]. level=4
Layer level=4: "dog"→"cog"; "log"→"cog"(visited). nextLayer=[cog]. level=5
Layer level=5: "cog"===end → return 5 ✓
```

Complexity:

- **Time:** $O(N \times L^2)$ roughly — N = words, ప్రతి word కి $L \times 26$ candidates, ప్రతి candidate build/compare $O(L)$. 26 constant. (Explicit graph build $O(N^2 \cdot L)$ కంటే మంచిది N పెద్దప్పుడు.)
- **Space:** $O(N \times L)$ — wordSet + visited + queue.
- **Optimization గమనిక: Bidirectional BFS** — begin నుండి, end నుండి ఏకకాలంలో BFS చేసి మధ్యలో కలిస్తే, search space చాలా తగ్గుతుంది (branching factor ను దాదాపు అర్థంగా). SSE interview లో "ఇంకా వేగంగా చేయగలవా?" అంటే ఇది చెప్పు.

గుర్తుంచుకోవాల్సినది:

"Word/state transformation + minimum steps + valid dictionary = BFS on implicit graph, neighbors generated on the fly." #433 (genes) ↔ #127 (words) కవలలు. Level-by-level BFS (nextLayer pattern) shortest distance ను శుభ్రంగా ఇస్తుంది. Follow-ups: **Word Ladder II** (#126 — అన్ని shortest paths, BFS + backtrack), **Open the Lock**. Explicit graph build అవసరం లేదు — neighbor generation + Set membership చాలు. Bidirectional BFS optimization గుర్తుంచుకో.

సాధారణ తప్పులు:

- **Level off-by-one:** answer = **words count** (edges+1). beginWord ను level 1 గా మొదలుపెట్టు; endWord చేరిన layer level. Edges (transformations) కావాలంటే **level-1**.
- **endWord wordList లో లేదు:** ముందే `return`; లేకపోతే BFS ఎప్పటికీ endWord చేరదు.
- **beginWord ను wordSet నుండి తీయకపోవడం:** beginWord wordList లో ఉండొచ్చు/ఉండకపోవచ్చు; visited లో పెట్టడం వల్ల మళ్ళీ process కాదు — safe.
- **Explicit graph build (N^2 comparisons):** పెద్ద wordList కి TLE. On-the-fly neighbor generation ($L \times 26$) వాడు.
- **Same-char skip:** `ch === word[i]` skip చేయకపోతే candidate == word అవుతుంది (visited వల్ల harmless కానీ అనవసరం).

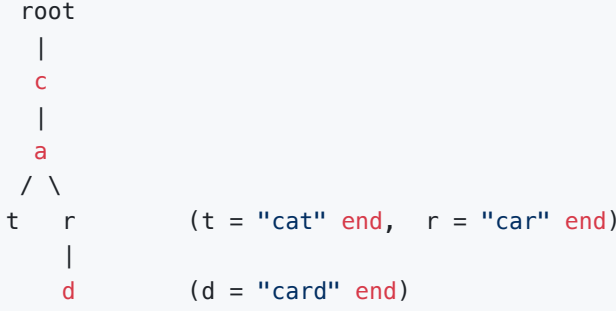
Pattern: Trie (Prefix Tree)

వివరణ

Trie (ఉచ్చారణ "try", **prefix tree** అని కూడా అంటారు) అంటే — strings ను వాటి **అక్షరాల ప్రకారం** నిల్వ చేసే ఒక tree. ప్రతి node ఒక అక్షరాన్ని represent చేస్తుంది; root నుండి ఒక node వరకు దారి (path) ఒక **prefix** ను ఏర్పరుస్తుంది.

Common prefix ఉన్న words ఒకే దారిని share చేస్తాయి — అదే Trie యొక్క మేజిక్.

ఉదాహరణ: "cat" , "car" , "card" – మూడూ c → a ను share చేస్తాయి, తర్వాత t (cat), r (car), r → d (card) గా విడిపోతాయి:



ప్రతి node కి కావాల్సినవి రెండే:

- **children** – ఏ అక్షరం → ఏ child node. {} object లేదా 26-size array.
- **isEnd (isWord)** – ఈ node దగ్గర ఒక పూర్తి word ముగుస్తుందా? ("car" ముగిసినా, "card" కోసం ఇంకా children ఉండొచ్చు – కాబట్టి ఈ flag అవసరం.)

మూడు core operations, అన్నీ **O(L)** (L = word/prefix length – ఎన్ని words ఉన్నా independent!):

- **insert(word)** – root నుండి, ప్రతి అక్షరానికి child లేకపోతే create చేస్తూ దిగు; చివరి node కి **isEnd = true**.
- **search(word)** – అక్షరాలు follow అవుతూ దిగు; ఏదైనా అక్షరం missing అయితే false; చివర్లో isEnd true అయితేనే word ఉంది.
- **startsWith(prefix)** – search లాంటిదే కానీ చివర్లో isEnd చూడనవసరం లేదు; path ఉంటే చాలు.

Real-life Scenario

Dictionary / phone contacts లో typing చేస్తున్నప్పుడు autocomplete. నువ్వు "car" type చేస్తుంటే, phone వెంటనే "card, care, careful, cartoon..." suggest చేస్తుంది. అది "car" prefix ఉన్న అన్ని words ను వెతకడం – Trie దీన్ని అమిత వేగంగా చేస్తుంది. ప్రతి typed అక్షరం tree లో ఒక అడుగు కిందికి; ఆ subtree లోని అన్ని words యే suggestions.

ఇంకో analogy: ఒక పెద్ద library లో పుస్తకాలు అమర్చడం. అన్ని "C" పుస్తకాలు ఒక అల్మారాలో, అందులో "Ca" ఒక అరలో, "Car" ఒక మూలలో. ఒక పుస్తకం వెతకాలంటే అక్షరం అక్షరం follow అయితే చాలు – మొత్తం library scan చేయనవసరం లేదు. Trie అదే – shared prefixes వల్ల space & time రెండూ ఆదా.

ఎలా గుర్తించాలి (Recognition Signals)

- **"Prefix", "startsWith", "autocomplete", "dictionary", "spell check"** అనే మాటలు → Trie.
- **చాలా words / strings మీద పదేపదే prefix / word lookups** – HashSet word lookup O(L) ఇస్తుంది కానీ **prefix** queries ను efficient గా చేయలేదు; Trie రెండింటినీ O(L) లో ఇస్తుంది.
- **Wildcard search (. any char), pattern matching over a dictionary** → Trie + DFS.
- **Grid/board లో ఒకేసారి చాలా words వెతకడం (Word Search II)** → Trie తో అన్ని words ను ఒకేసారి match చేయడం (ఒక్కో word కి separate search బదులు).

Node structure + insert / search template

```
// ప్రతి node: children map + word end flag
class TrieNode {
  constructor() {
    this.children = {}; // అక్షరం -> TrieNode ('a' -> node). Object సులభం, flexible.
    this.isEnd = false; // ఇక్కడ ఒక పూర్తి word ముగుస్తుందా?
  }
}

class Trie {
  constructor() {
    this.root = new TrieNode(); // root ఏ అక్షరాన్ని hold చేయదు - కేవలం starting point
  }

  insert(word) {
    let node = this.root;
    for (const ch of word) {
      // ఈ అక్షరానికి దారి లేకపోతే కొత్త node సృష్టించు
      if (!node.children[ch]) node.children[ch] = new TrieNode();
      node = node.children[ch]; // ఒక అడుగు కిందికి
    }
    node.isEnd = true; // చివరి node = word ముగింపు
  }

  // ఒక string ను follow చేసి, చివరి node ను return (దారి తెగితే null)
  _walk(str) {
    let node = this.root;
    for (const ch of str) {
      if (!node.children[ch]) return null; // అక్షరం missing -> దారి లేదు
      node = node.children[ch];
    }
    return node;
  }

  search(word) {
    const node = this._walk(word);
    return node !== null && node.isEnd; // దారి ఉంది + word ముగుస్తుంది
  }

  startsWith(prefix) {
    return this._walk(prefix) !== null; // దారి ఉంటే చాలు (isEnd అవసరం లేదు)
  }
}
```

Complexity

- **insert / search / startsWith:** $O(L)$ — L = word/prefix length. **ఎన్ని words store చేసినా** ఈ ops L మీదే ఆధారపడతాయి (word count మీద కాదు) — ఇదే Trie గొప్పతనం.
- **Space:** $O(\text{total characters across all words} \times \text{alphabet})$ worst case. Shared prefixes space ఆదా చేస్తాయి. Object children (`{}`) alphabet size మేర memory కాకుండా ఉన్న అక్షరాలకే కేటాయిస్తుంది (26-array కంటే memory-friendly).

గుర్తుంచుకో: HashSet strings ను store చేస్తుంది కానీ **prefix relationships** ను కోల్పోతుంది. Trie ఆ structure ను నిలుపుతుంది – అందుకే prefix queries, wildcard search, autocomplete లకు అనువైనది. isEnd flag ను ఎప్పుడూ మర్చిపోకు: "car" ఒక word, కానీ "ca" (కేవలం prefix) word కాదు – ఈ తేడాను isEnd చెబుతుంది.

10. Implement Trie (Prefix Tree) (LeetCode #208) – Medium

సమస్య: ఒక Trie class ను implement చెయ్యి, ఈ methods తో:

- `insert(word)` – word ను trie లో చేర్చు.
- `search(word)` – word సరిగ్గా trie లో ఉంటే `true` (గతంలో insert చేసినది).
- `startsWith(prefix)` – trie లో prefix తో మొదలయ్యే ఏదైనా word ఉంటే `true`.

Constraints: `1 <= word.length, prefix.length <= 2000`; lowercase English letters; మొత్తం 3×10^4 calls వరకు.

ఉదాహరణ:

```
insert("apple");
search("apple"); → true
search("app"); → false    ("app" ను insert చేయలేదు, కేవలం prefix)
startsWith("app"); → true  ("apple" ఆ prefix తో మొదలవుతుంది)
insert("app");
search("app"); → true     (ఇప్పుడు insert చేశాం)
```

ఎలా ఆలోచించాలి:

ఇది Trie primer లో నేర్చుకున్నదాన్ని నేరుగా అమలు చేసే problem – "Trie ను స్వయంగా కట్టు." కీలక design decision: **node structure**. ప్రతి node కి (1) children (అక్షరం → child node), (2) isEnd flag. isEnd ఎందుకు అవసరం? "apple" insert చేశాక, `search("app")` false ఇవ్వాలి – కానీ `a→p→p` దారి trie లో ఉంది! తేడా: "app" ముగింపు node కి `isEnd=false`. అదే flag "ఇది నిజమైన word ఆ, కేవలం prefix ఆ?" అని చెబుతుంది.

మూడు operations ఒకే pattern: root నుండి అక్షరం అక్షరం walk. `insert` – missing child ను create చేస్తూ దిగి, చివర `isEnd=true`. `search` / `startsWith` – walk; ఏ అక్షరం missing అయితే false. తేడా చివర్లో: `search` కి `isEnd` కూడా true కావాలి; `startsWith` కి దారి ఉంటే చాలు. `_walk` helper తో duplicate code తగ్గించొచ్చు.

Optimal Approach:

Insight: Node = {children, isEnd}. అన్ని ops $O(L)$ walk. `search` vs `startsWith` తేడా = చివర్లో isEnd check ఉండా.

Plan:

1. `TrieNode`: children = {}, `isEnd` = false. `Trie`: root = new `TrieNode()`.
2. `insert`: walk + missing child create; చివర `isEnd=true`.
3. `_walk(str)`: follow; missing → null; చివరి node return.

4. search: `_walk` node && node.isEnd . startsWith: `_walk` `!== null` .

Solution (JavaScript):

```
class TrieNode {
  constructor() {
    this.children = {}; // char -> TrieNode
    this.isEnd = false; // ఈ node దగ్గర ఒక word ముగుస్తుందా
  }
}

class Trie {
  constructor() {
    this.root = new TrieNode(); // root అక్షరం hold చేయదు
  }

  insert(word) {
    let node = this.root;
    for (const ch of word) {
      if (!node.children[ch]) node.children[ch] = new TrieNode(); // దారి లేకపోతే కట్టు
      node = node.children[ch];
    }
    node.isEnd = true; // word ముగింపును గుర్తు పెట్టు
  }

  // str ను follow చేసి చివరి node; దారి తెగితే null (search, startsWith రెండింటికీ)
  _walk(str) {
    let node = this.root;
    for (const ch of str) {
      if (!node.children[ch]) return null; // అక్షరం లేదు → దారి తెగింది
      node = node.children[ch];
    }
    return node;
  }

  search(word) {
    const node = this._walk(word);
    return node !== null && node.isEnd; // దారి + నిజమైన word ముగింపు
  }

  startsWith(prefix) {
    return this._walk(prefix) !== null; // దారి ఉంటే చాలు
  }
}
```

Dry Run:

```
insert("apple"):
  root→a(new)→p(new)→p(new)→l(new)→e(new); e.isEnd=true
search("apple"): walk a-p-p-l-e ✓, node.isEnd=true → true ✓
search("app"):   walk a-p-p ✓, కానీ ఆ 'p' node.isEnd=false → false ✓
startsWith("app"): walk a-p-p ✓, node≠null → true ✓
insert("app"):   walk a-p-p (అన్నీ ఇప్పటికే ఉన్నాయి); రెండో 'p'.isEnd=true
search("app"):   walk a-p-p, ఇప్పుడు isEnd=true → true ✓
```

Complexity:

- **Time:** `insert` , `search` , `startsWith` ప్రతిదీ $O(L)$ (L = word/prefix length). **Store చేసిన words సంఖ్యతో సంబంధం లేదు.**
- **Space:** $O(N \times L)$ worst case (N words, shared prefixes ఆదా చేస్తాయి). ప్రతి కొత్త అక్షరం $\leq$ ఒక కొత్త node.

గుర్తుంచుకోవాల్సినది:

Trie = $O(L)$ insert/search/prefix, word count తో సంబంధం లేకుండా. ఈ base implementation తర్వాతి రెండు problems (#211 wildcard, #212 Word Search II) కి foundation — దీన్ని కళ్ళు మూసుకుని రాయగలగాలి. కీలక design: **children map + isEnd flag.** `isEnd` లేకపోతే prefix ను word నుండి వేరు చేయలేం. Object `{}` children flexible & memory-friendly (26-array కంటే). Autocomplete, spell-checker, IP routing (longest prefix match) — అన్నిటికీ ఇదే base.

సాధారణ తప్పులు:

- **isEnd flag లేకపోవడం:** flag లేకుండా, `search("app")` ("apple" మాత్రమే insert చేశాక) తప్పుగా true ఇస్తుంది — prefix ను word గా భ్రమిస్తుంది. `isEnd` ప్రాణం.
- **search & startsWith ను ఒకేలా చూడటం:** `search` కి చివర్లో `isEnd` తప్పనిసరి; `startsWith` కి అవసరం లేదు. ఈ తేడా మర్చిపోతే tests fail.
- **root వద్ద అక్షరం store చేయడం:** root ను ఖాళీగా (కేవలం starting point) ఉంచు; మొదటి అక్షరం root యొక్క child.
- **Missing child ను handle చేయకపోవడం:** `walk` లో `node.children[ch]` లేకపోతే వెంటనే null/false. Check మర్చిపోతే `undefined.children` → crash.

11. Design Add and Search Words Data Structure (LeetCode #211) — Medium

సమస్య: ఒక data structure design చెయ్యి, ఇది words add చేయడం మరియు వాటిని match చేయడం support చేస్తుంది:

- `addWord(word)` — word ను structure లో చేర్చు.
- `search(word)` — word structure లో ఉంటే `true`. కానీ ఇక్కడ మెలిక: word లో `'.'` (dot) ఉండొచ్చు, అది ఏదైనా ఒక్క అక్షరాన్ని match చేస్తుంది (wildcard).

Constraints: $1 \leq \text{word.length} \leq 25$; `addWord` లో lowercase మాత్రమే; `search` లో lowercase లేదా `'.'` ; dots ఉన్న words కి $\text{word.length} \leq 2$ డాట్‌ల పరిమితి కాదు కానీ dots ≤ 2 (LeetCode). మొత్తం 10^4 calls.

ఉదాహరణ:

```
addWord("bad"); addWord("dad"); addWord("mad");
search("pad") → false
search("bad") → true
```

```
search(".ad") → true    ('.' → b/d/m ఏదైనా; "bad","dad","mad" match)
search("b..") → true    ("bad" match: b, .=a, .=d)
```

ఎలా ఆలోచించాలి:

Base — ఇది Trie (#208). addWord = insert, exact search = trie search. కొత్త సవాలు: **'.'** wildcard. సాధారణ search లో ప్రతి అక్షరానికి ఒకే child కి వెళ్తాం. కానీ **'.'** వచ్చినప్పుడు — ఏ child అయినా match కావచ్చు! కాబట్టి ఒక్క దారి కాదు, **అన్ని children దారులా try చేయాలి**, ఏదైనా ఒకటి success అయితే చాలు.

"అన్ని branches try చేసి, ఏదైనా ఒకటి success ఆ?" — ఇది **DFS / backtracking** signal. Search ను recursive గా రాయ: dfs(node, i) — word యొక్క i -th అక్షరం దగ్గర, ప్రస్తుత node. అక్షరం normal అయితే ఆ ఒక్క child కి దిగు. **'.'** అయితే, **ప్రతి child** మీద dfs(child, i+1) try చేసి, ఏదైనా true ఇస్తే true. i === word.length (అక్షరాలు అయిపోయాయి) అయితే node.isEnd return.

Optimal Approach:

Insight: Trie + recursive search. Normal char → single child descend. **'.'** → అన్ని children మీద DFS branch (ఏదైనా match చాలు).

Plan:

1. TrieNode {children, isEnd}; addWord = standard insert.
2. search(word) → dfs(root, 0) .
3. dfs(node, i) : i===len → node.isEnd . ch = word[i] . **'.'** అయితే అన్ని children మీద dfs(child, i+1) , ఏదైనా true → true. లేకపోతే child లేకపోతే false, ఉంటే dfs(child, i+1) .

Solution (JavaScript):

```
class TrieNode {
  constructor() {
    this.children = {};
    this.isEnd = false;
  }
}

class WordDictionary {
  constructor() {
    this.root = new TrieNode();
  }

  addWord(word) {
    // standard trie insert
    let node = this.root;
    for (const ch of word) {
      if (!node.children[ch]) node.children[ch] = new TrieNode();
      node = node.children[ch];
    }
    node.isEnd = true;
  }

  search(word) {
    // recursive DFS: node = ప్రస్తుత స్థానం, i = word లో index
    const dfs = (node, i) => {
```

```

    if (i === word.length) return node.isEnd; // అక్షరాలు అయిపోయాయి → word ముగుస్తుంది

    const ch = word[i];
    if (ch === '.') {
        // wildcard: ప్రతి child ను try చెయ్యి; ఏదైనా ఒకటి match అయితే చాలు
        for (const key in node.children) {
            if (dfs(node.children[key], i + 1)) return true;
        }
        return false; // ఏ child దారీ పని చేయలేదు
    }
    // normal అక్షరం: ఆ ఒక్క child లేకపోతే false, ఉంటే దాని లోకి దిగు
    if (!node.children[ch]) return false;
    return dfs(node.children[ch], i + 1);
};

return dfs(this.root, 0);
}
}

```

Dry Run: add "bad","dad","mad"; search(".ad")

```

trie root children: b, d, m (ప్రతిది → a→d(isEnd))
dfs(root,0): ch='.' → అన్ని children try:
  dfs(b-node,1): ch='a' → b's child 'a' ✓ → dfs(a-node,2):
    ch='d' → child 'd' ✓ → dfs(d-node,3): i===3===len → d.isEnd=true → true
  మొదటి branch (b) true → return true ✓

search("b.."):
dfs(root,0): 'b' → b-node; dfs(_,1): '.' → b's children {a}:
  dfs(a-node,2): '.' → a's children {d}: dfs(d-node,3): i===len, isEnd=true → true ✓
search("pad"): dfs(root,0) 'p' → root కి 'p' child లేదు → false ✓

```

Complexity:

- **Time:** addWord $O(L)$. search normal word $O(L)$. Dots ఉంటే worst case $O(26^d \times L)$ — d = dots సంఖ్య (ప్రతి dot వద్ద అన్ని children branch). అన్నీ dots అయితే $O(26^L)$, కానీ practice లో trie sparse కాబట్టి చాలా తక్కువ. LeetCode constraints (dots ≤ 2) manageable.
- **Space:** $O(N \times L)$ trie + $O(L)$ recursion depth.

గుర్తుంచుకోవాల్సినది:

Trie + wildcard/pattern search = DFS over the trie (branch at wildcards). కీలక insight: normal char = deterministic single step; wildcard = **try all children, any success wins** = DFS with OR. ఇదే idea: pattern matching over a dictionary, regex-lite over strings. Base Trie ను DFS తో augment చేయడం ఒక reusable trick — Word Search II (#212) లోనూ ఇదే. "అన్ని అవకాశాలు try చేసి ఏదైనా ఒకటి?" $\Rightarrow$ DFS/backtracking.

సాధారణ తప్పులు:

- **Wildcard వద్ద ఒక్క child మాత్రమే try చేయడం:** '.' కి అన్ని children మీద recurse చేయాలి; ఒకటి try చేసి అగితే valid matches miss అవుతాయి.

- **Base case లో isEnd మర్చిపోవడం:** $i === len$ వద్ద కేవలం `true` return చేస్తే prefix ను word గా భ్రమిస్తుంది; `node.isEnd` return చెయ్యాలి.
- **Iterative search ను wildcard తో బలవంతం చేయడం:** wildcards branching కావాలి — iterative single-pointer walk సరిపోదు; recursion/stack అవసరం.
- **for..in తో isEnd వంటి extra property కలవడం:** ఇక్కడ children ప్రత్యేక object (isEnd వేరే field) కాబట్టి safe. Node structure ను children/flag కలిపి ఒకే flat object చేస్తే `for..in` లో flag కూడా iterate అవుతుంది — వేరు చేయడం మంచిది (ఈ code లా).

12. Word Search II (LeetCode #212) — Hard

సమస్య: $m \times n$ board (అక్షరాలతో) మరియు `words` list ఇస్తారు. Board లో **నిర్మించగలిగే** అన్ని words ను return చెయ్యి. ఒక word ను నిర్మించడం = **పక్కపక్క (horizontal/vertical) cells** ద్వారా వరుసగా అక్షరాలు కలపడం; **ఒకే cell ను ఒక word లో రెండుసార్లు వాడకూడదు**.

Constraints: $1 \leq m, n \leq 12$; $1 \leq words.length \leq 3 \times 10^4$; word length ≤ 10 ; lowercase.

ఉదాహరణ:

```
board = [
  ["o","a","a","n"],
  ["e","t","a","e"],
  ["i","h","k","r"],
  ["i","f","l","v"]
]
words = ["oath","pea","eat","rain"]
Output: ["oath","eat"]    ("oath": o(0,0)→a(0,1)→t(1,1)→h(2,1); "eat" board లో ఉంది)
```

ఎలా ఆలోచించాలి:

Naive ఆలోచన: ప్రతి word కీ, Word Search I (#79) లాగా board అంతా DFS చేసి వెతకడం. కానీ words 3×10^4

ఉన్నాయి! ప్రతి word కీ separate board DFS → అపారమైన repeated work (చాలా words ఒకే prefix `"ea..."` share చేస్తాయి, కానీ ప్రతిదాన్ని విడిగా వెతుకుతాం). చాలా నెమ్మది.

కీలక insight: **అన్ని words ను ఒక Trie లో పట్టు**, తర్వాత board ను **ఒకేసారి** DFS చెయ్యి, trie ద్వారా guide అవుతూ. Board లో ఒక cell నుండి DFS చేస్తూ, ప్రస్తుత path ను **trie లో parallel గా follow** చెయ్యి. ప్రస్తుత అక్షరం trie లో child గా లేకపోతే — ఆ దారిలో ఏ word కూడా match కాదు → వెంటనే **prune** (అగు). ఇది common prefixes ను share చేస్తుంది: "oath", "oats" రెండూ `o→a` ను ఒకేసారి explore. Trie యొక్క prefix pruning ఇక్కడ మేజిక్.

DFS backtracking: cell ను visit చేసేటప్పుడు temporary గా mark ('#'), children explore, తర్వాత **restore** (అదే word లో reuse కాకుండా, కానీ వేరే words కీ available). Trie node లో పూర్తి word (`node.word`) కనిపిస్తే result కీ add; duplicate రాకుండా `node.word = null` చేసి "consumed" గా mark.

Optimal Approach:

Insight: అన్ని words → ఒక Trie. Board DFS ను trie తో guide చెయ్యి — trie లో child లేని అక్షరం వద్ద prune. Word-end node కనిపిస్తే collect. Prefix sharing + pruning వల్ల భారీ speedup.

Plan:

1. అన్ని words తో trie build; ప్రతి word end node కి `node.word = word` (full word store).
2. ప్రతి cell `(r,c)` నుండి `dfs(r, c, root)`.
3. `dfs` : bounds/visited/trie-child లేకపోతే return. `next = node.children[ch]` . `next.word` ఉంటే result కి push, `next.word = null` (dedup). Cell '#' mark; 4 దిక్కులు `dfs(_, next)` ; cell restore.

Solution (JavaScript):

```
function findWords(board, words) {
  // 1) అన్ని words ను ఒక trie లో పెట్టు (child object; end node లో పూర్తి word)
  const root = {};
  for (const word of words) {
    let node = root;
    for (const ch of word) {
      node[ch] = node[ch] || {}; // child లేకపోతే కట్టు
      node = node[ch];
    }
    node.word = word; // end node వద్ద పూర్తి word ను నిల్వ చెయ్యి
  }

  const rows = board.length, cols = board[0].length;
  const result = [];

  const dfs = (r, c, node) => {
    if (r < 0 || r >= rows || c < 0 || c >= cols) return; // హద్దు దాటింది
    const ch = board[r][c];
    if (ch === '#' || !node[ch]) return; // ఇప్పటికే వాడారో, లేదా trie లో ఈ దారి లేదు (PRUNE)

    const next = node[ch]; // trie లో ఒక అడుగు కిందికి
    if (next.word) { // ఇక్కడ ఒక పూర్తి word ముగిసింది
      result.push(next.word);
      next.word = null; // duplicate రాకుండా consumed గా mark
    }

    board[r][c] = '#'; // visited గా mark (అదే word లో reuse అవు)
    dfs(r + 1, c, next); // కింద
    dfs(r - 1, c, next); // పైన
    dfs(r, c + 1, next); // కుడి
    dfs(r, c - 1, next); // ఎడమ
    board[r][c] = ch; // ⚠️ backtrack: restore (వేరే words కి available)
  };

  for (let r = 0; r < rows; r++) {
    for (let c = 0; c < cols; c++) {
      dfs(r, c, root); // ప్రతి cell ను possible starting point గా try
    }
  }
  return result;
}
```

Dry Run: board `[["o","a"],["e","t"]]` , words `["oat","eat","oa"]`

trie: root → o→a(word="oa")→t(word="oat"), e→a→t(word="eat")
`dfs(0,0='o', root)`: root['o'] ఉంది → next=o-node (word? none).

```

mark (0,0)='#'. explore neighbors with o-node:
dfs(1,0='e', o-node): o-node['e']? లేదు → PRUNE (return)
dfs(0,1='a', o-node): o-node['a'] ఉంది → next=a-node, word="oa" →
    result=["oa"], clear word. mark(0,1)='#'.
    dfs neighbors with a-node: dfs(1,1='t', a-node): a-node['t'] ఉంది →
        word="oat" → result=["oa","oat"]. ... restore.
restore (0,1)='a'. restore (0,0)='o'.
dfs(1,0='e', root): root['e'] → e→a→t → "eat" → result=["oa","oat","eat"] ✓
(మిగతా starts prune అవుతాయి)

```

Complexity:

- **Time:** $O(m \times n \times 4 \times 3^{(L-1)})$ worst case — ప్రతి starting cell నుండి DFS, ప్రతి అడుగులో ≤ 3 కొత్త దిక్కులు (వచ్చిన దిక్కు తప్ప), $\text{depth} \leq L$ (max word length). Trie pruning వల్ల practice లో దీనికంటే చాలా వేగం. Trie build $O(\text{total chars in words})$.
- **Space:** $O(\text{total chars in words})$ — trie + $O(L)$ recursion depth. Board in-place (extra visited array అవసరం లేదు).

గుర్తుంచుకోవాల్సినది:

"Board/grid లో చాలా words ను ఒకేసారి వెతకాలి = Trie + DFS backtracking." ఒక్కో word కి separate search ($O(W \times mn \times 3^L)$) బదులు, అన్ని words ను ఒక trie లో పెట్టి board ను ఒకేసారి DFS చేయడం — common prefixes share అవుతాయి, dead branches prune అవుతాయి. ఇది "Trie + Graph DFS" రెండు patterns కలయిక — ఈ guide యొక్క capstone. కీలక tricks: (1) end node లో పూర్తి word store, (2) found word ను `null` చేసి dedup, (3) '#' తో visited mark + backtrack restore.

సాధారణ తప్పులు:

- **Backtrack restore మర్చిపోవడం:** cell ను '#' చేశాక తిరిగి original అక్షరానికి restore చేయకపోతే, తర్వాతి words ఆ cell ను వాడలేవు → words miss. `board[r][c] = ch` తప్పనిసరి.
- **Duplicate words:** ఒకే word board లో రెండు దారుల్లో match కావచ్చు. `next.word = null` (found మీద) లేకపోతే result లో duplicates. (ప్రత్యామ్నాయం: `Set` వాడు.)
- **ప్రతి word కి separate DFS (Trie లేకుండా):** 3×10^4 words కి TLE. Trie యొక్క ఏకైక ప్రయోజనం ఇక్కడ prefix sharing + pruning.
- **Prune మర్చిపోవడం:** `!node[ch]` వద్ద return చేయకపోతే, trie లో లేని దారుల్లోనూ DFS కొనసాగుతుంది → exponential blowup. Trie pruning ఈ solution యొక్క గుండె.
- **(Advanced) Leaf pruning:** ఒక word దొరికాక, ఆ trie leaf ఖాళీ అయితే తీసేయడం (`delete`) ఇంకా వేగం — పెద్ద inputs కి interviewer మెచ్చే optimization.

ముగింపు: Pattern Recognition Cheat-Sheet

ఈ 12 problems వెనుక ఉన్నది కొన్ని core patterns మాత్రమే. Problem statement లో ఈ **signals** కనిపిస్తే, ఏ pattern వాడాలో వెంటనే గుర్తుపట్టు:

Signal (problem లో కనిపించేది)	Pattern	ఉదాహరణ problems
Grid + "island / region / connected / surrounded / flood"	DFS/BFS flood fill (implicit graph)	Number of Islands (#200), Surrounded Regions (#130)
"Deep copy / clone" a graph	DFS/BFS + Map<original, clone>	Clone Graph (#133)
Elements మధ్య ratio / conversion / relationship	Weighted graph, path product	Evaluate Division (#399)
"Can finish / possible?" + prerequisites/dependencies	Topo sort — cycle detection (Kahn count)	Course Schedule (#207)
"Valid order / sequence" + dependencies	Topo sort — collect order (Kahn)	Course Schedule II (#210)
"Minimum moves / fewest steps", equal-cost moves	BFS (levels = distance)	Snakes & Ladders (#909)
"Minimum transformations", one small change/step, valid set	BFS on implicit state graph	Genetic Mutation (#433), Word Ladder (#127)
"prefix / startsWith / autocomplete / dictionary"	Trie	Implement Trie (#208)
Trie + wildcard / pattern match	Trie + DFS (branch at wildcard)	Add & Search Words (#211)
Board లో చాలా words ఒకేసారి వెతకడం	Trie + DFS backtracking	Word Search II (#212)

అన్నిటికీ వర్తించే 6 బంగారు నియమాలు:

1. **"Graph" అనే మాట లేకపోయినా graph ను చూడు** — grid, strings, ratios, dependencies అన్నీ nodes + edges. మొదటి ప్రశ్న ఎప్పుడూ: **"nodes ఏవి? edges ఏవి?"**
2. **DFS vs BFS ఎంపిక:** "shortest / minimum steps / fewest" ⇒ **BFS** (unweighted). "అన్నీ చూడాలి / connectivity / path ఉందా / clone" ⇒ **DFS** (recursion సులభం).
3. **Visited-marking traversal యొక్క ప్రాణం** — లేకపోతే infinite loop (cycles) / TLE. BFS లో **enqueue వద్దే** mark చెయ్యి (dequeue కాదు). Grid లో in-place mark ('#' / '0') space ఆదా.
4. **"Order + dependency" ⇒ Topological Sort.** Kahn (BFS + indegree): processed count === total కాకపోతే **cycle**. Edge direction (prereq → dependent) జాగ్రత్తగా set చెయ్యి.
5. **"Prefix / dictionary / autocomplete" ⇒ Trie** — insert/search O(L), word count తో సంబంధం లేదు. isEnd flag తో word ను prefix నుండి వేరు చెయ్యి. Wildcard ⇒ DFS branch.
6. **Backtracking లో ఎప్పుడూ restore చెయ్యి** — '#' mark చేశాక (Word Search II) తిరిగి original అక్షరానికి తేవడం మర్చిపోకు; disconnected components కోసం అన్ని nodes/cells నుండి traversal try చెయ్యి.

గుర్తుంచుకో: Graph & Trie అంటే బట్టి కాదు – **input ను graph గా అనువదించడం + సరైన traversal**

ఎంచుకోవడం (DFS/BFS/Topo) + visited discipline. ఈ 3 patterns (Graph traversal, Topological Sort, Trie) తెలిస్తే, LeetCode 150 లోని ఈ 12 మాత్రమే కాదు, వీటి variants వందల్లో solve చేయగలవు. ప్రతి problem ని "ఏ signal → ఏ pattern" అని అనువదించడం అలవాటు చేసుకో. అదే SSE interview లో గెలిపిస్తుంది.